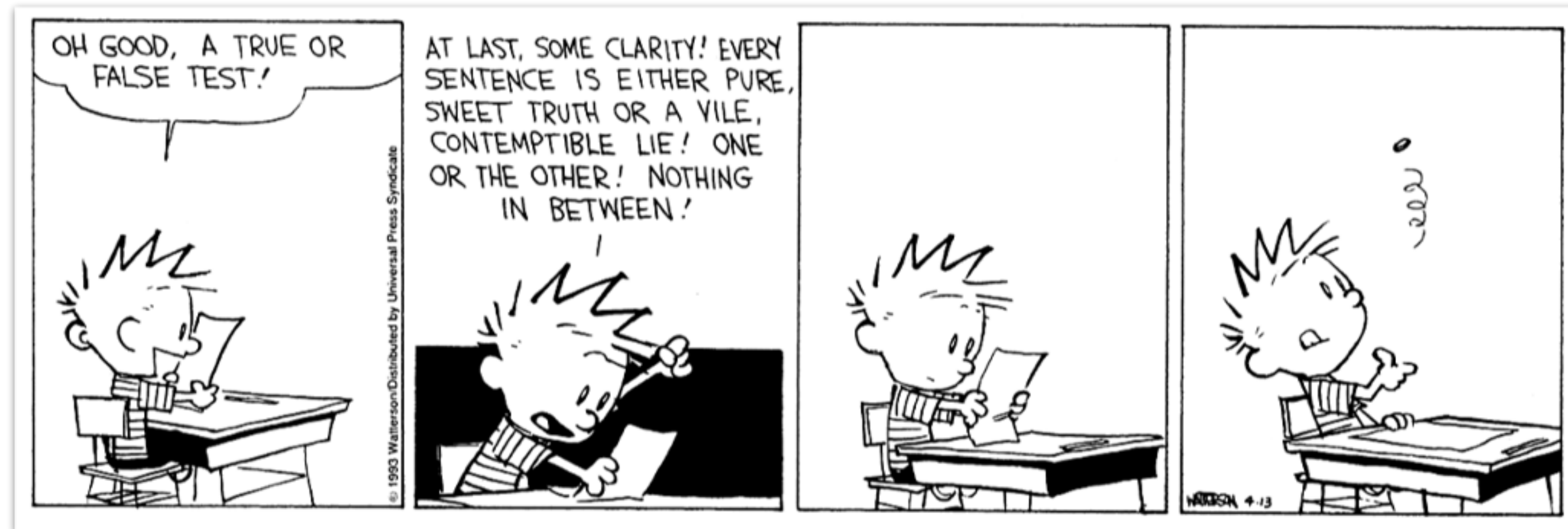


Model comparison



02/13/2026

Logistics

Project proposal

Project proposal

**Members of teams
will all get the
same grade!**

**maximum 3 team
members**

IT'S TIME FOR A...
GROUP ASSIGNMENT!!



Didn't attend
any group
meetings



Doesn't
understand
the material



Gave the
presentation
but obviously
didn't know
what he was
even saying



Who is
this guy

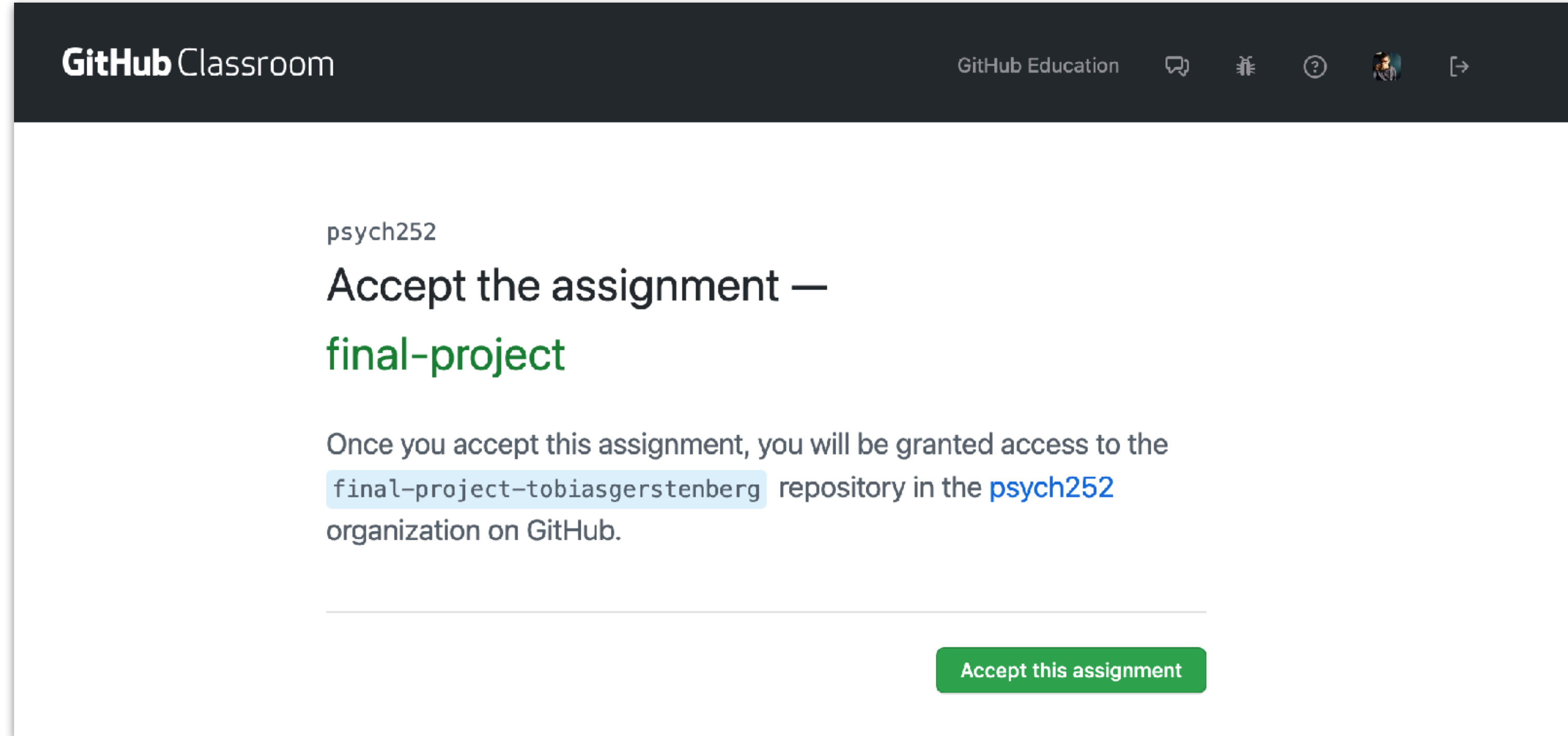


"You can
use my
printer"



Did all the
research, wrote
paper, composed
presentation

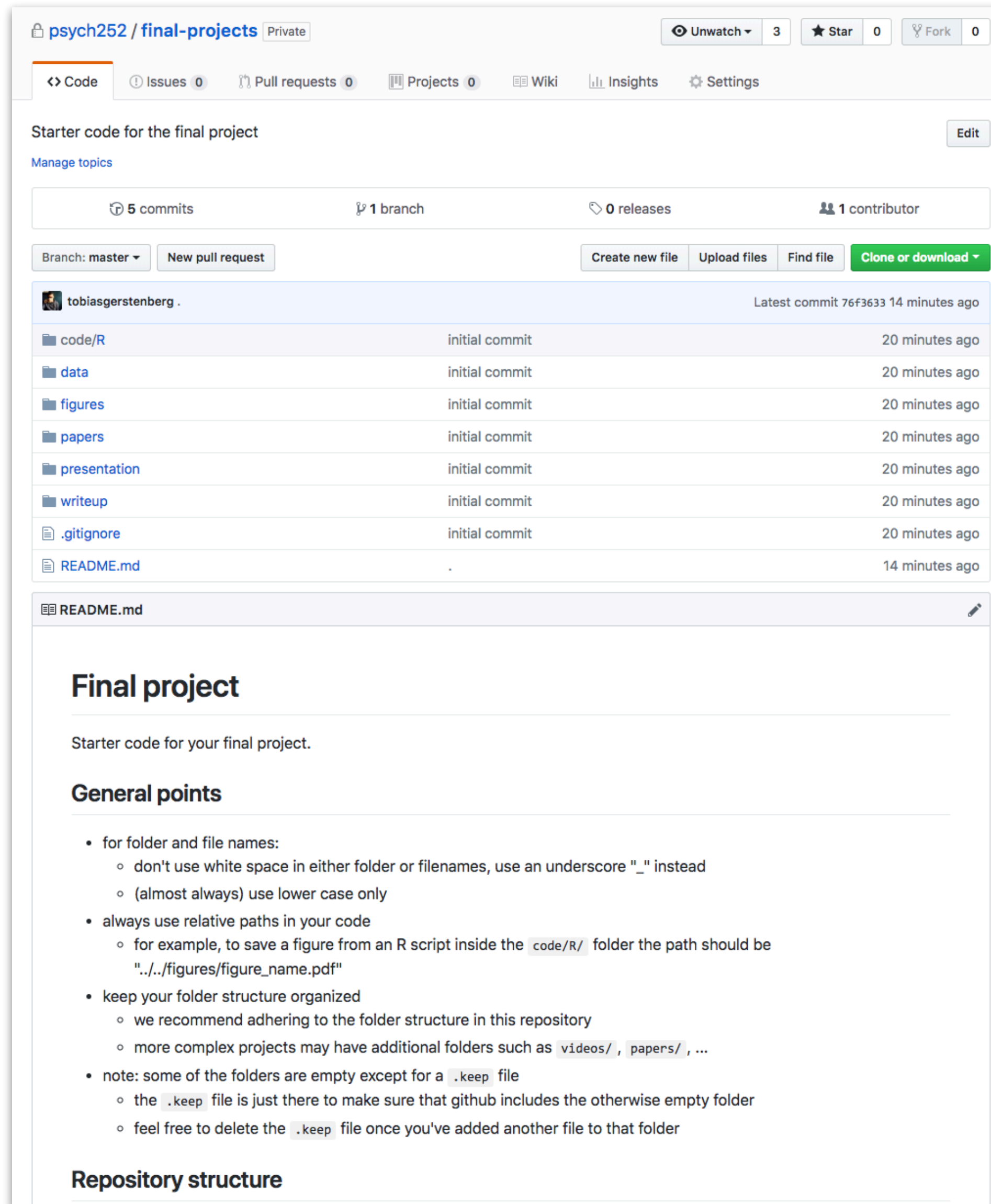
Project proposal



if you work as a team, have one person accept the assignment and the others can then join that repository

<https://tinyurl.com/psych252project26>

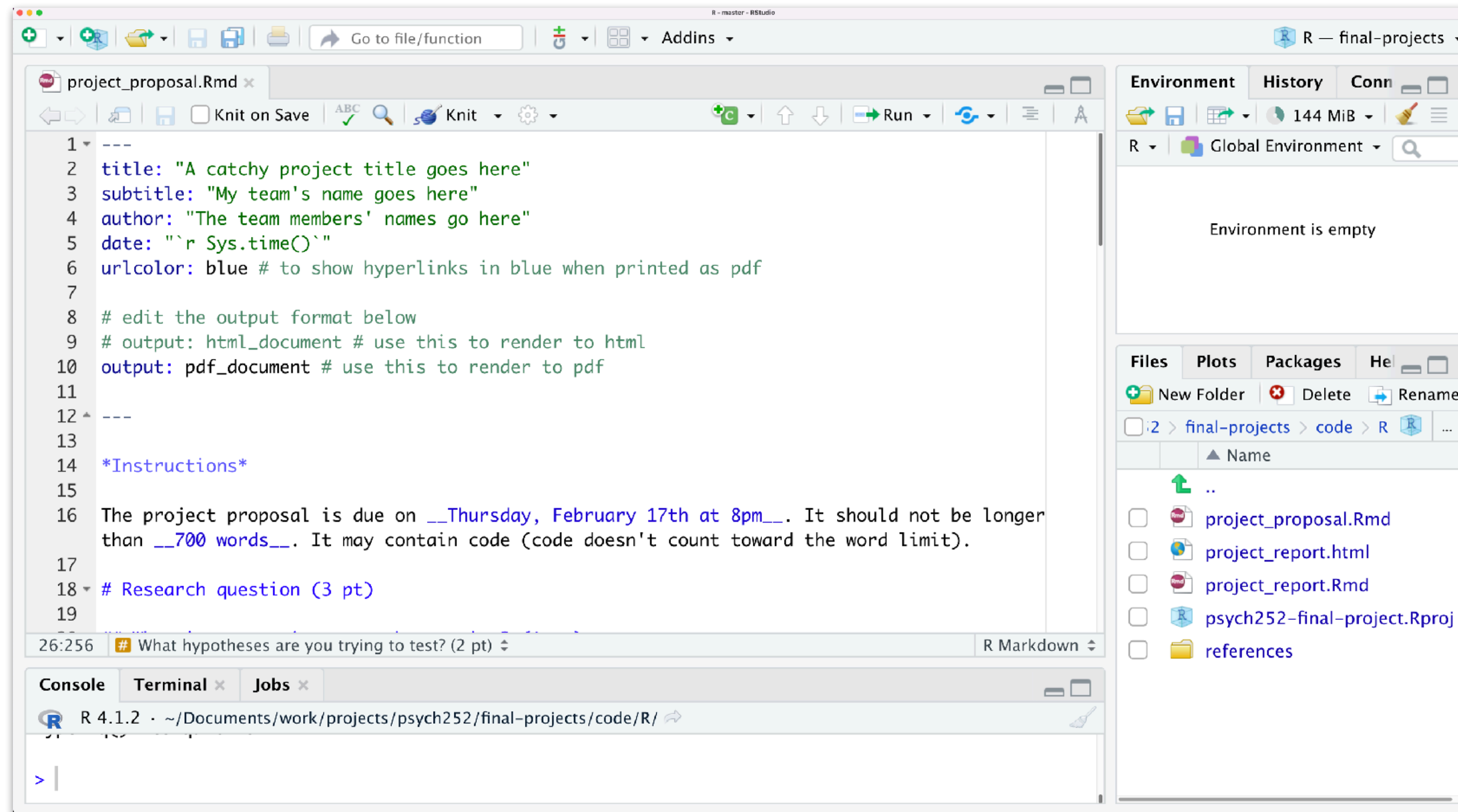
Project proposal



- each team will have their own private github repository
- all work on your final project should happen within this repository
- you can get **github** help in section
- post on Ed Discussion in case you experience any problems getting set up (use **final_project** tag)

Project proposal

RMarkdown template



psych252/final-projects/code/R/project_proposal.Rmd

Upload the
pdf to canvas

Project proposal

The project proposal is due on
Thursday, February 19th at 8pm

Project proposal

Files

Switch to Old Files Page



All My Files

+ Folder

Upload

Search files

Search files...

Search

Enter at least 2 characters to search

Statistical Methods for Behavioral and Social Sciences > final_project_samples > project_proposal

0 selected



☐	Name ^	Created ↕	Last Modified ↕	Modified By ↕	Size ↕	Rights ↕	Status	Actions
☐	anjie_cao.pdf	Friday	Monday		236 KB			
☐	danyang_fan.pdf	Friday	Monday		259 KB			
☐	exams_project_proposal.pdf	Friday	Monday		326 KB			
☐	hannah_marshall.pdf	Friday	Yesterday		238 KB			
☐	madi_jamie_catie.pdf	Friday	Monday		247 KB			
☐	meditation_project_proposal.pdf	Friday	Monday		1.7 MB			
☐	sleep_project_proposal.pdf	Friday	Yesterday		343 KB			
☐	tunes_project_proposal.pdf	Friday	Friday		262 KB			

458.2 MB of 5.2 GB used

< 1-8 of 8 >

examples of project proposals from prior years

Plan for today

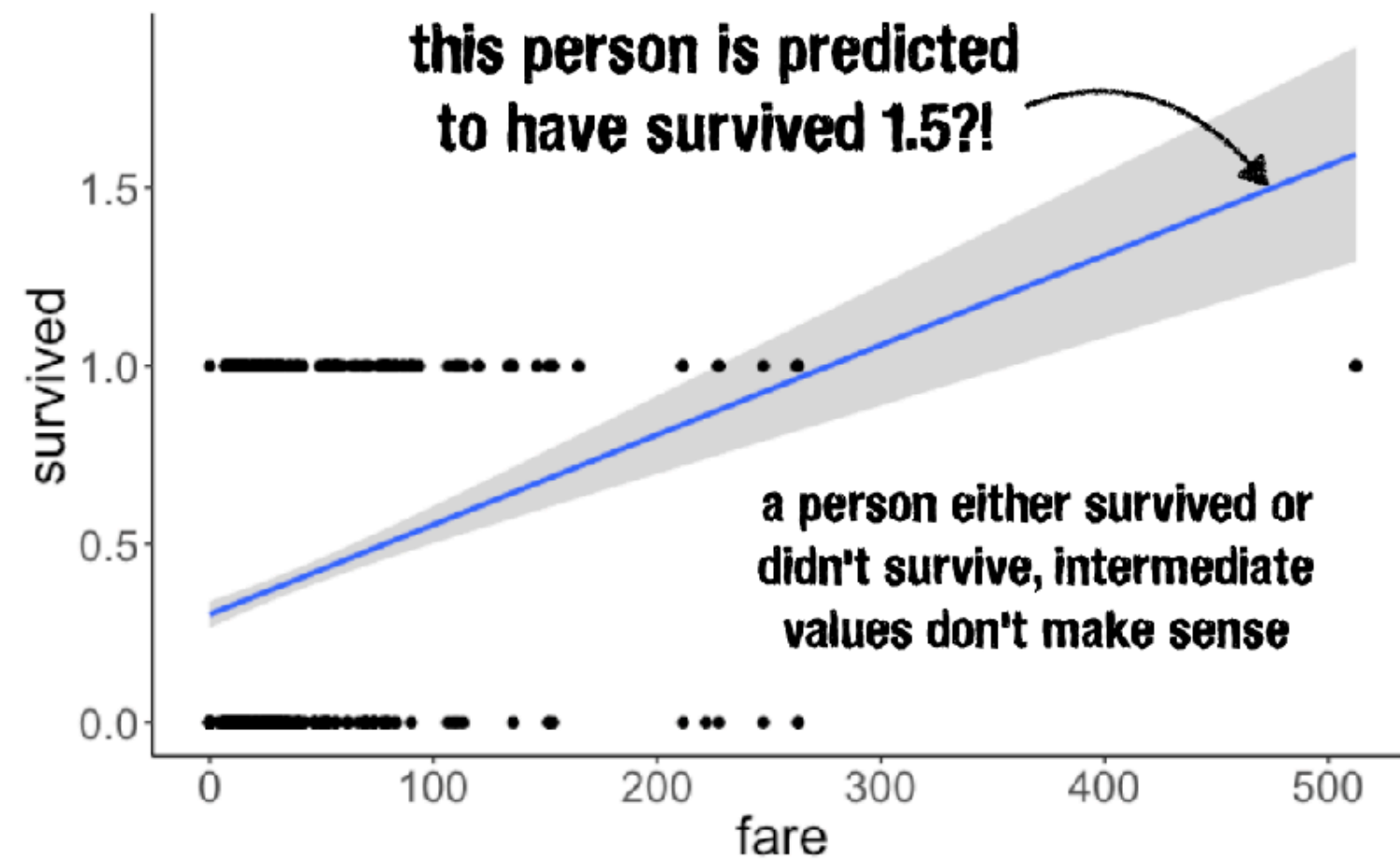
- Quick recap
- Power analysis
 - Making decisions
 - Calculating power
 - Effect sizes
 - Determining sample size
- Model comparison
 - Cross-validation
 - AIC and BIC

Quick recap

Quick recap: Logistic regression

Is there a relationship between fare and survived?

```
fit.lm = lm(formula = survived ~ 1 + fare, data = df.titanic)
```



22

Logit transform

$$\pi_i = b_0 + b_1 \cdot X_i + e_i \quad \text{predict the probability of } Y$$

$$\pi_i = P(Y_i = 1)$$

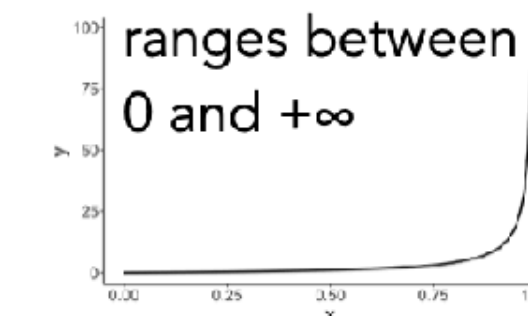
Step 1: Calculate the "odds"

$$\frac{P(Y_i = 1)}{P(Y_i = 0)} = \frac{\pi_i}{1 - \pi_i}$$

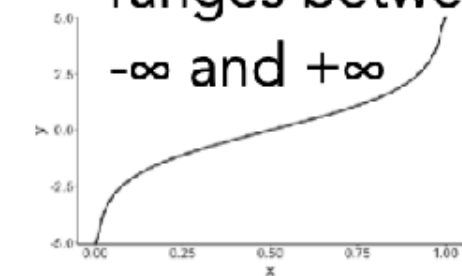
Step 2: Take the (natural) log

$$\ln\left(\frac{\pi_i}{1 - \pi_i}\right) = b_0 + b_1 \cdot X_i + e_i$$

we need to transform the dependent variable so that it can take any value between $-\infty$ and $+\infty$ (we can then transform it back into a probability later)



ranges between $-\infty$ and $+\infty$



25

Fitting a logistic regression in R

```
1 fit.glm = glm(formula = survived ~ 1 + fare,
2               family = "binomial",
3               data = df.titanic)
4
5 fit.glm %>% summary()
```

```
Call:
glm(formula = survived ~ 1 + fare, family = "binomial", data = df.titanic)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-2.4906  -0.8878  -0.8531   1.3429   1.5942

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.941330   0.095129  -9.895 < 2e-16 ***
fare         0.015197   0.002232   6.810 9.79e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

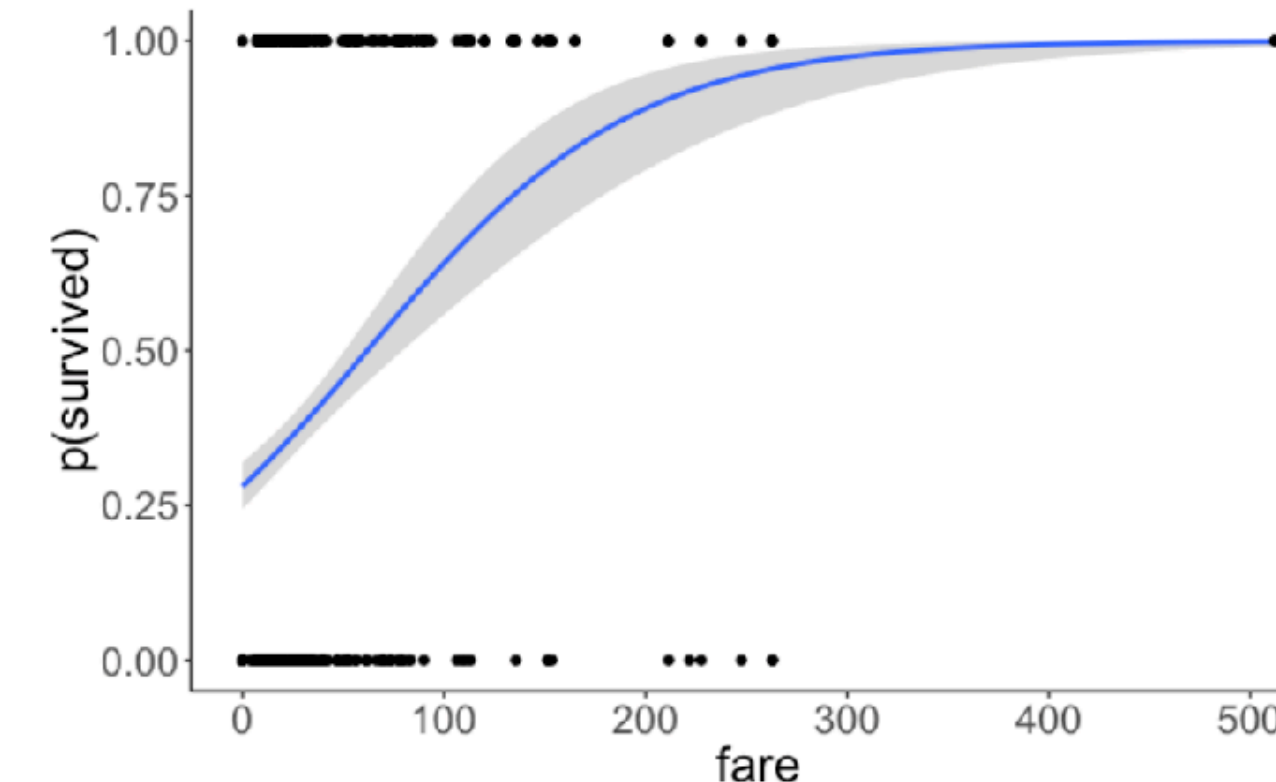
    Null deviance: 1186.7  on 890  degrees of freedom
Residual deviance: 1117.6  on 889  degrees of freedom
AIC: 1121.6

Number of Fisher Scoring iterations: 4
```

27

Visualize the model's predictions

```
1 ggplot(data = df.titanic,
2         mapping = aes(x = fare,
3                       y = survived)) +
4   geom_smooth(method = "glm",
5               method.args = list(family = "binomial")) +
6   geom_point() +
7   labs(y = "p(survived)")
```

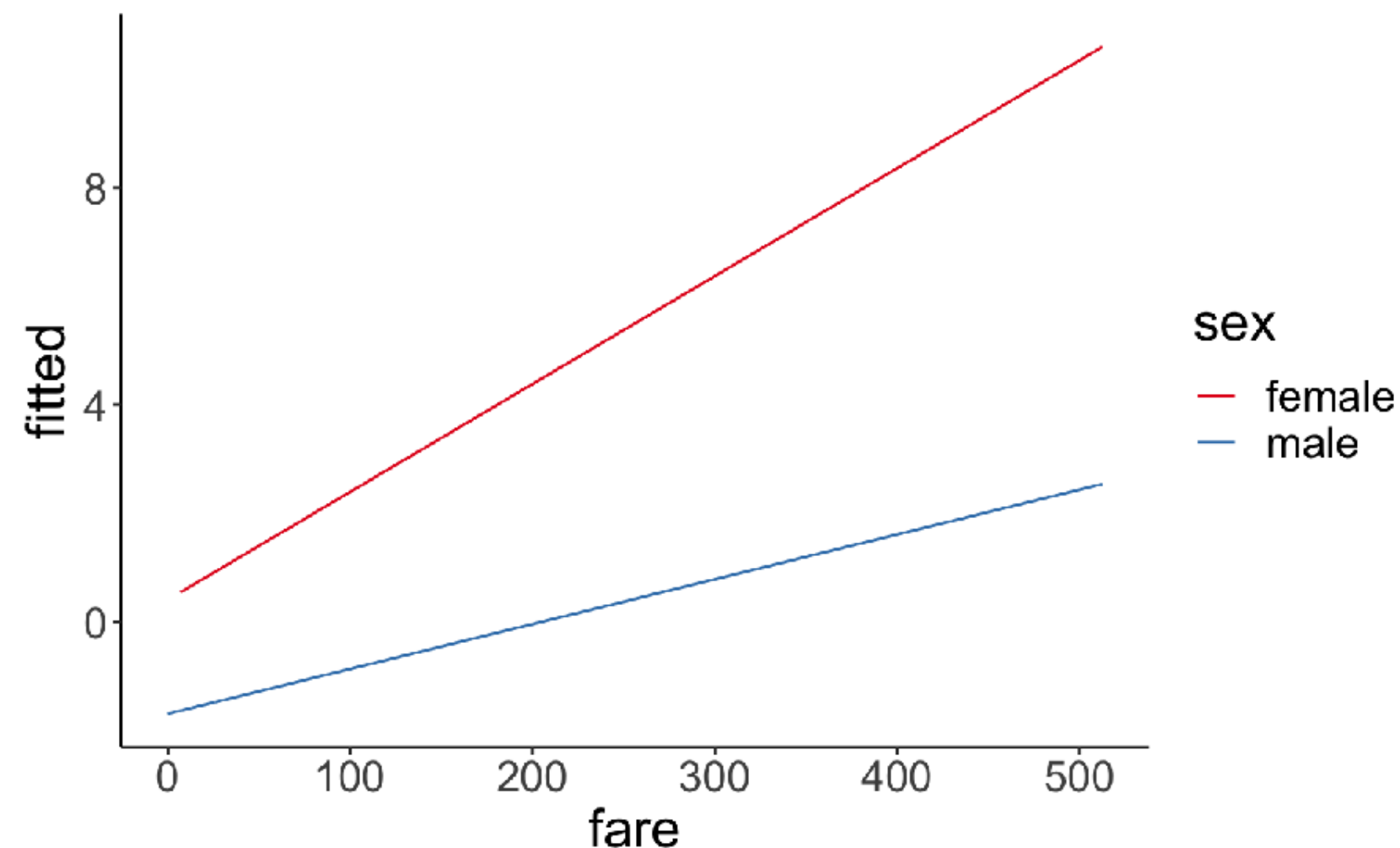


28

12

Quick recap: Logistic regression

```
1 fit.glm3 = glm(formula = survived ~ 1 + sex * fare,  
2               family = "binomial",  
3               data = df.titanic)  
4  
5 fit.glm3 %>%  
6   summary()
```



**lines in log odds
space**

Call:
glm(formula = survived ~ 1 + sex * fare, family = "binomial",
data = df.titanic)

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.6280	-0.6279	-0.5991	0.8172	1.9288

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.408428	0.189999	2.150	0.031584 *
sexmale	-2.099345	0.230291	-9.116	< 2e-16 ***
fare	0.019878	0.005372	3.701	0.000215 ***
sexmale:fare	-0.011617	0.005934	-1.958	0.050252 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

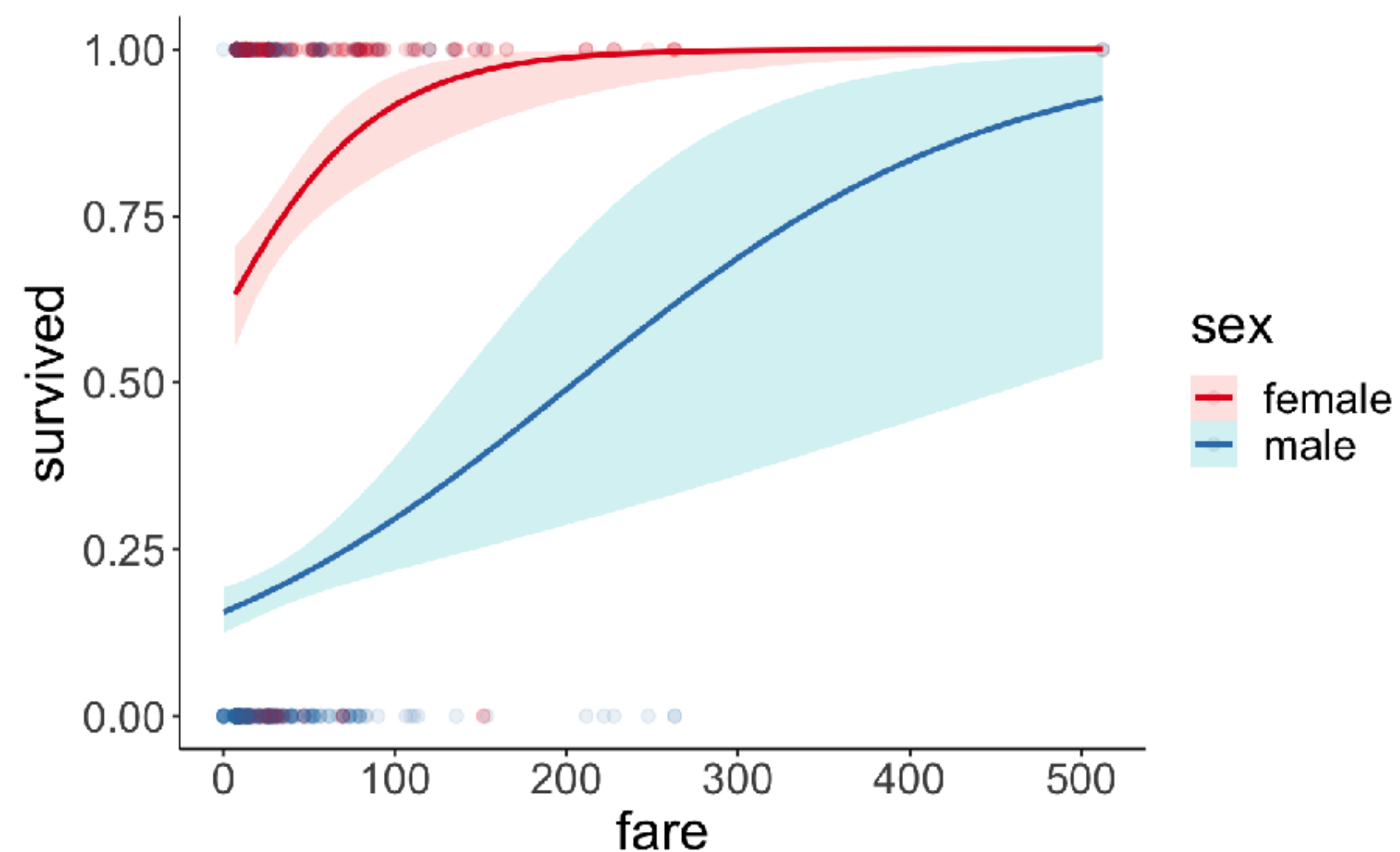
(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1186.66 on 890 degrees of freedom
Residual deviance: 879.85 on 887 degrees of freedom
AIC: 887.85

Number of Fisher Scoring iterations: 5

Quick recap: Logistic regression

```
1 fit.glm3 = glm(formula = survived ~ 1 + sex * fare,  
2               family = "binomial",  
3               data = df.titanic)  
4  
5 fit.glm3 %>%  
6   summary()
```



**after inverse logit
transformation**

Call:
glm(formula = survived ~ 1 + sex * fare, family = "binomial",
data = df.titanic)

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.6280	-0.6279	-0.5991	0.8172	1.9288

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.408428	0.189999	2.150	0.031584 *
sexmale	-2.099345	0.230291	-9.116	< 2e-16 ***
fare	0.019878	0.005372	3.701	0.000215 ***
sexmale:fare	-0.011617	0.005934	-1.958	0.050252 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1186.66 on 890 degrees of freedom
Residual deviance: 879.85 on 887 degrees of freedom
AIC: 887.85

Number of Fisher Scoring iterations: 5

Quick recap: Fitting and reporting models

Simulating a logistic regression

```

1 # make example reproducible
2 set.seed(1)
3
4 # set parameters
5 sample_size = 1000
6 b0 = 0
7 b1 = 1
8
9 # generate data
10 df.data = tibble(
11   x = rnorm(n = sample_size),
12   y = b0 + b1 * x,
13   p = inv.logit(y)
14 ) %>% mutate(response = rbinom(n(), size = 1, p = p))
15
16 # fit model
17 fit = glm(formula = response ~ 1 + x,
18           family = "binomial",
19           data = df.data)
20
21 # model summary
22 fit %>% summary()

```

set some parameters

linear model (y is in log odds)

transform into probability

randomly draw response

fit a logistic regression

summarize the result

45

Assessing the model fit

actual value

predicted value

$$\text{log-likelihood} = \sum_{i=1}^n [Y_i \cdot \ln(P(Y_i)) + (1 - Y_i) \cdot \ln(1 - P(Y_i))]$$

- calculate the probability of the observed response
- take the log of these probabilities
- sum them up to get the log-likelihood of the data (given the model)

response	p(Y = 1)	p(Y = response)	log(p(Y = response))
1	0.34	0.34	-1.07
0	0.53	0.47	-0.75
1	0.30	0.30	-1.20
1	0.81	0.81	-0.22
1	0.56	0.56	-0.58
0	0.30	0.70	-0.36
1	0.60	0.60	-0.52
1	0.65	0.65	-0.43
1	0.62	0.62	-0.48
0	0.41	0.59	-0.54

47

Testing hypotheses

```

joint_tests()

1 glm(formula = survived ~ 1 + sex + fare,
2     family = "binomial",
3     data = df.titanic) %>%
4     joint_tests()

model term df1 df2 T.ratio p.value
sex      1 Inf 201.881 <.0001
fare     1 Inf 23.869 <.0001

```

not quite the same value

```

anova(test = "LRT")

1 # fit compact model
2 fit.compact = glm(formula = survived ~ 1 + fare,
3                   family = "binomial",
4                   data = df.titanic)
5
6 # fit augmented model
7 fit.augmented = glm(formula = survived ~ 1 + sex + fare,
8                     family = "binomial",
9                     data = df.titanic)
10
11 # likelihood ratio test
12 anova(fit.compact, fit.augmented, test = "LRT")

```

Analysis of Deviance Table

```

Model 1: survived ~ 1 + fare
Model 2: survived ~ 1 + sex + fare
Df Sum of Sq Residual Deviance Pr(>Chi)
1    889    1117.57      233.26 < 2.2e-16 ***
2    888    884.31      233.26 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

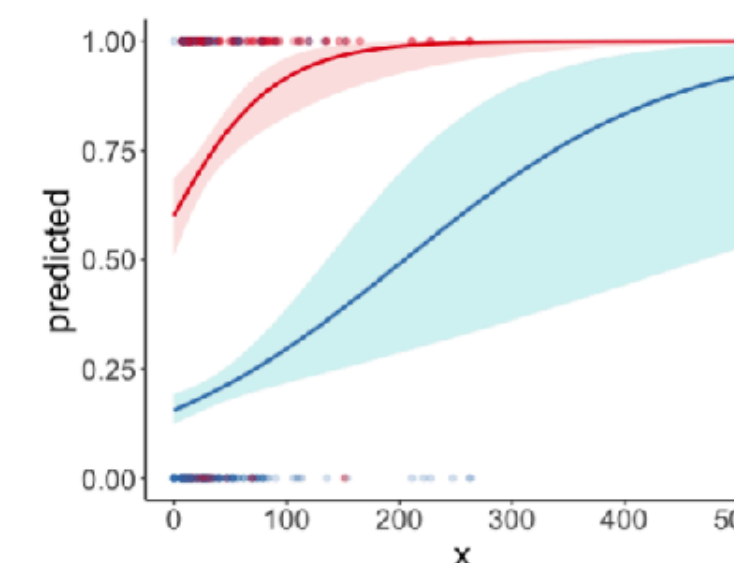
```

<https://stats.stackexchange.com/questions/400101/using-emmeans-with-clmm-to-look-at-joint-effects>

51

Reporting results

- Visualize the data
- Show a table with the regression results
- Report significance of different factors
- Interpreting parameter estimates is tricky -- probably best to report probabilities for a few example cases



Predicted values of survived
x = fare

sex = female

x	Predicted	SE	95% CI
0	0.50	0.13	[0.51, 0.69]
100	0.92	0.42	[0.83, 0.96]
200	0.99	0.95	[0.93, 1.00]
300	1.00	1.28	[0.97, 1.00]
400	1.00	2.02	[0.99, 1.00]
500	1.00	2.55	[1.00, 1.00]

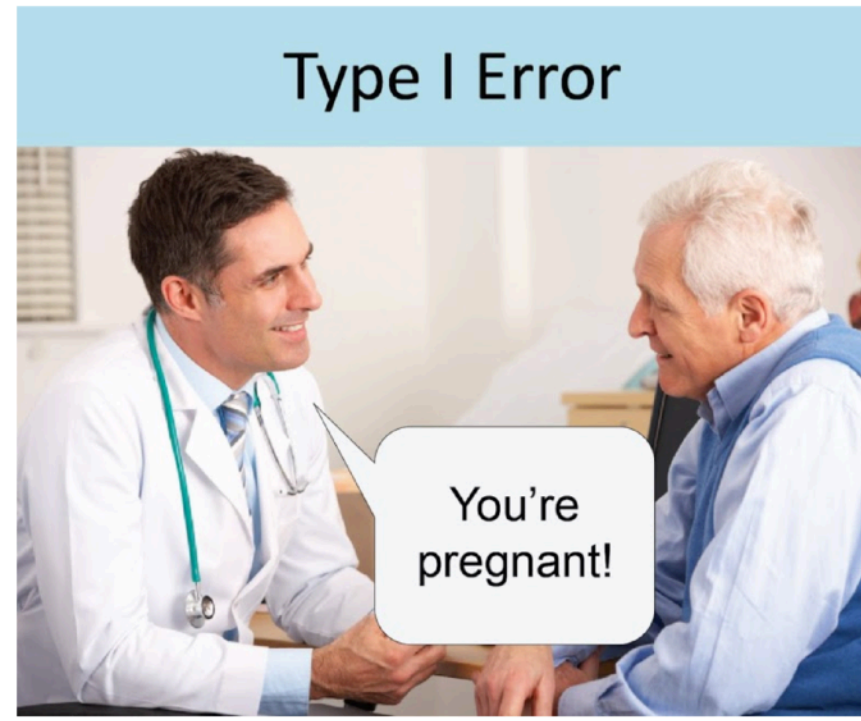
sex = male

x	Predicted	SE	95% CI
0	0.16	0.13	[0.12, 0.19]
100	0.30	0.21	[0.22, 0.39]
200	0.49	0.44	[0.23, 0.70]
300	0.59	0.63	[0.25, 0.90]
400	0.83	0.94	[0.44, 0.97]
500	0.92	1.13	[0.53, 0.99]

54

15

Quick recap: Power analysis



H_0 : Not pregnant. H_1 : Pregnant.

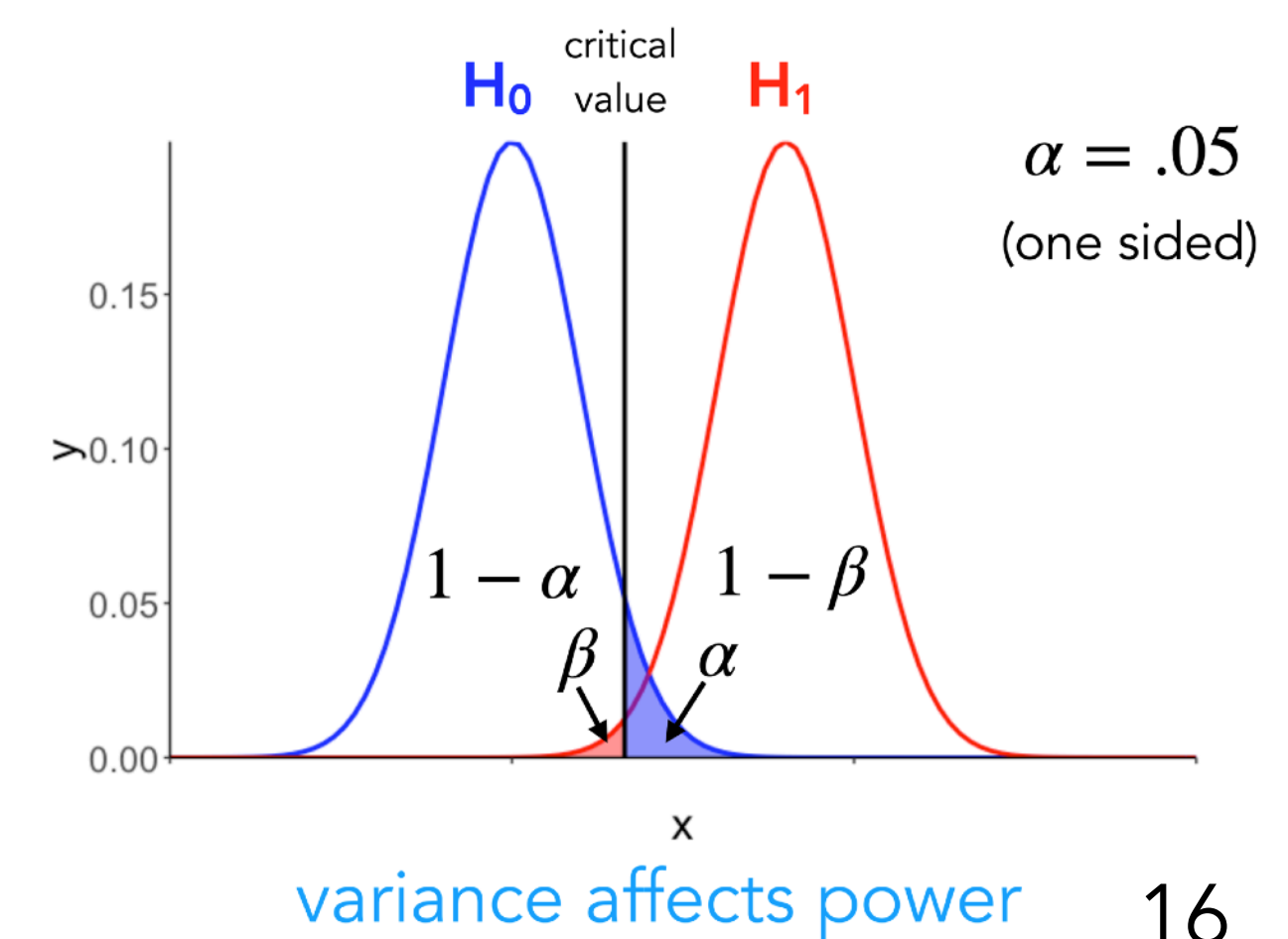
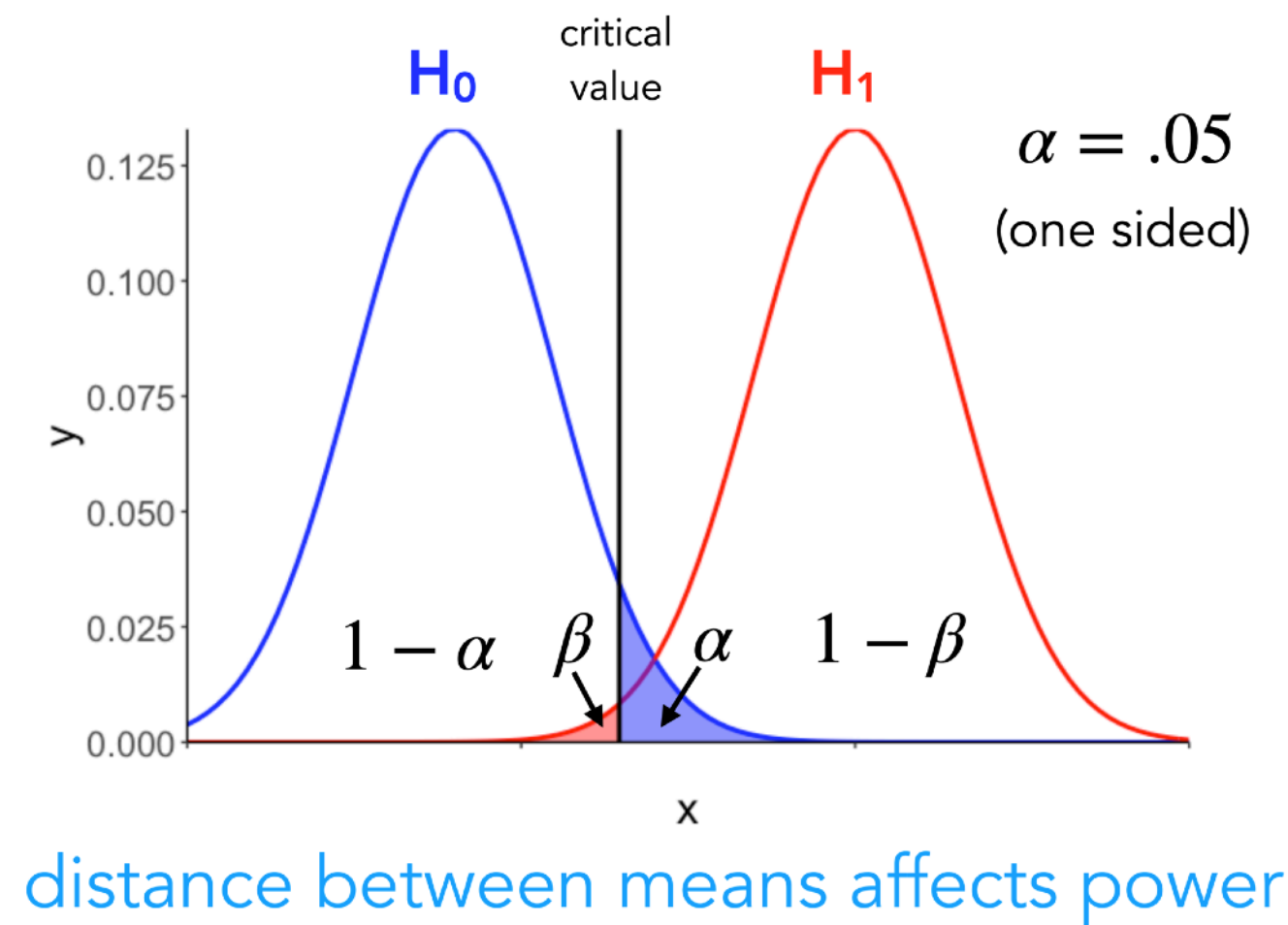
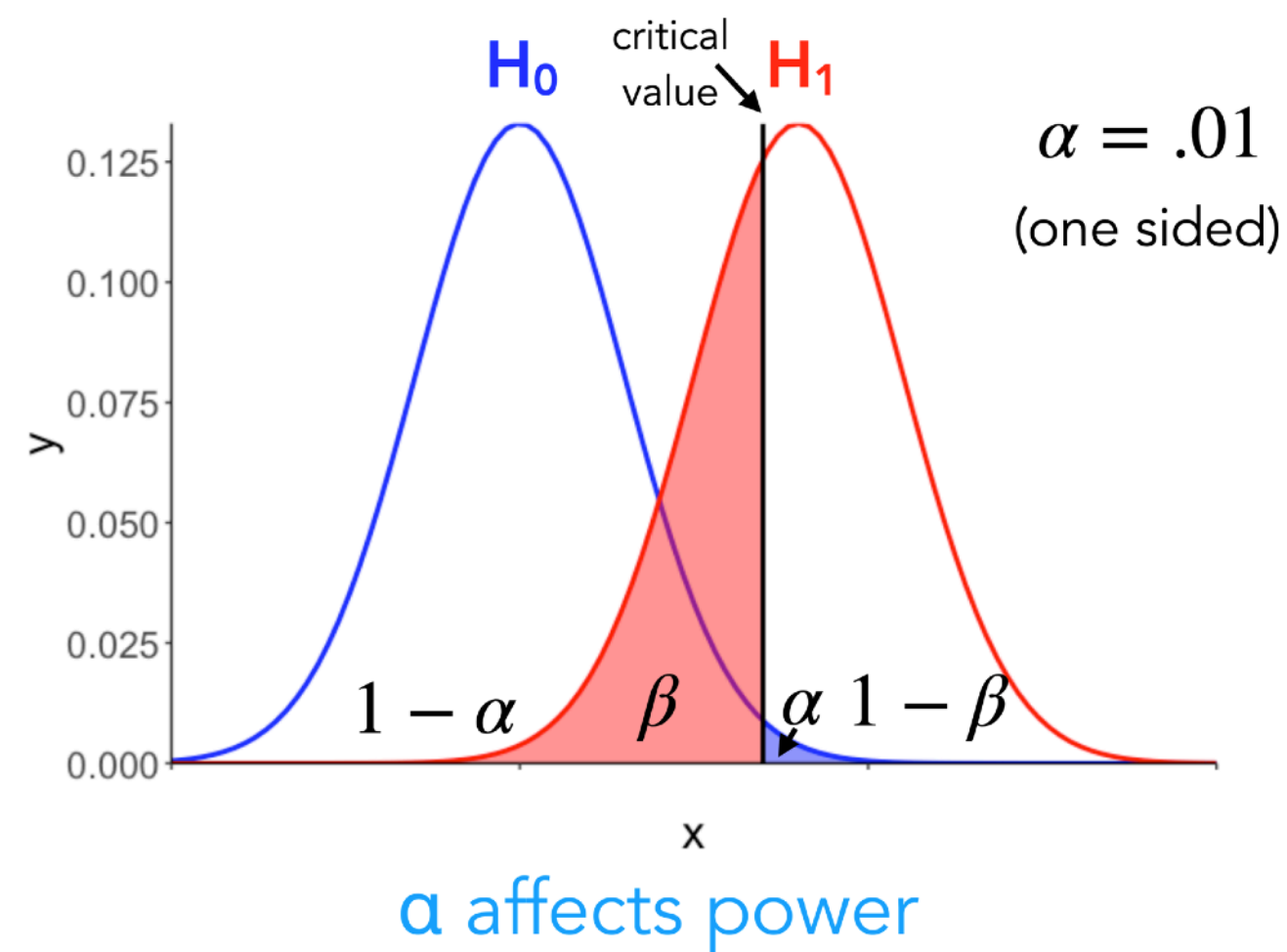
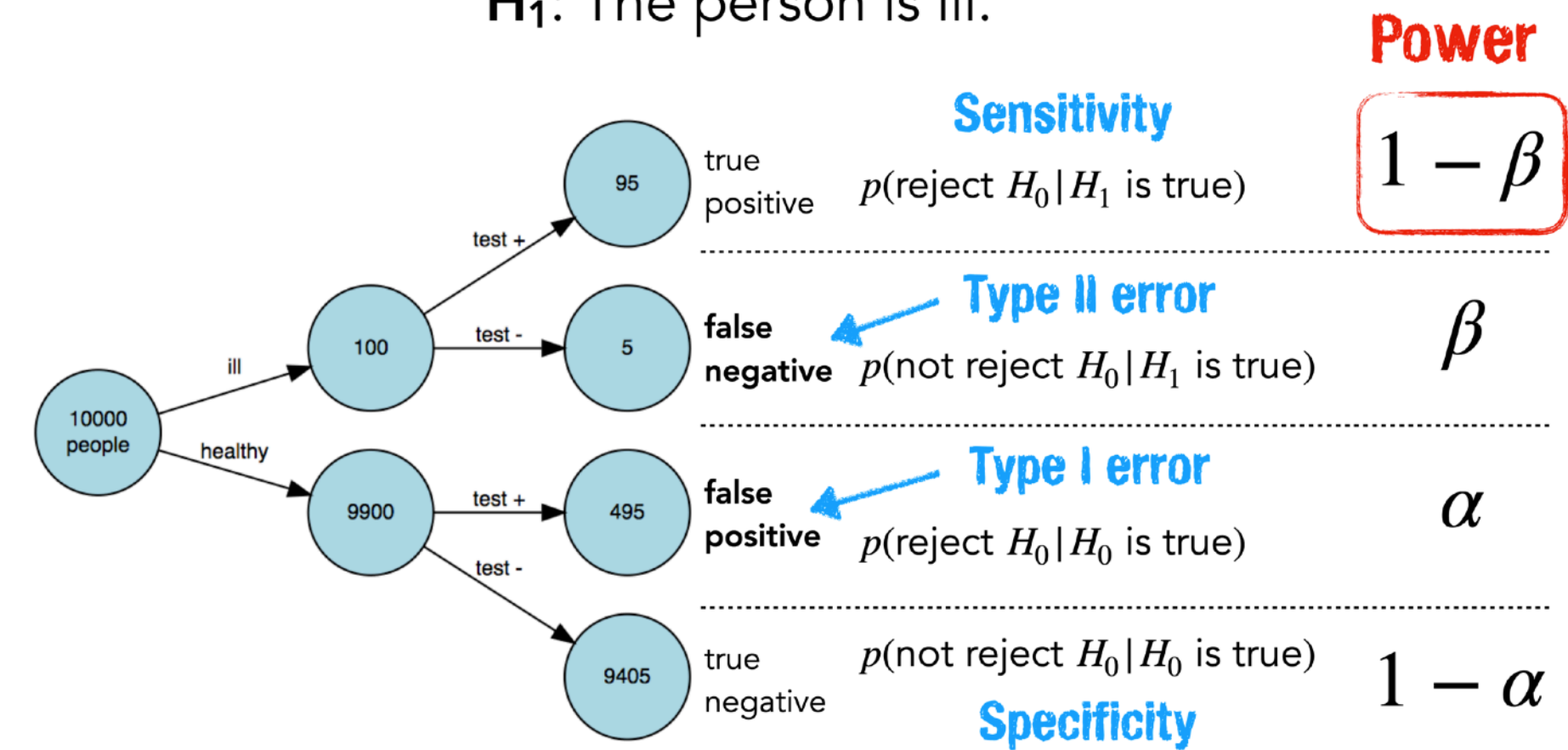
Type I Error: Falsely rejecting the null hypothesis (even though it is true).

Type II Error: Failing to reject the null hypothesis (even though it is false).

Clue guide to probability

H_0 : The person is healthy.

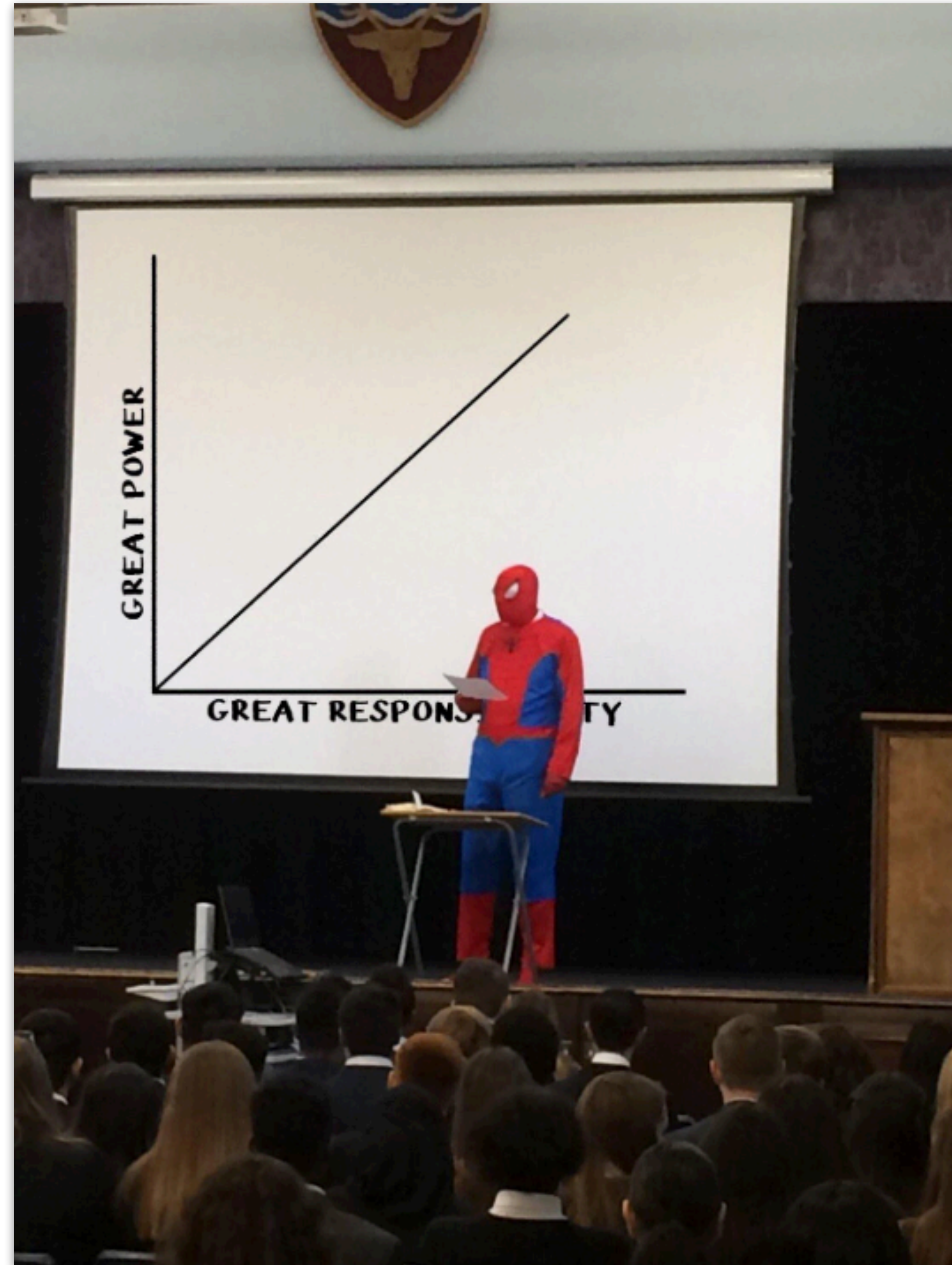
H_1 : The person is ill.



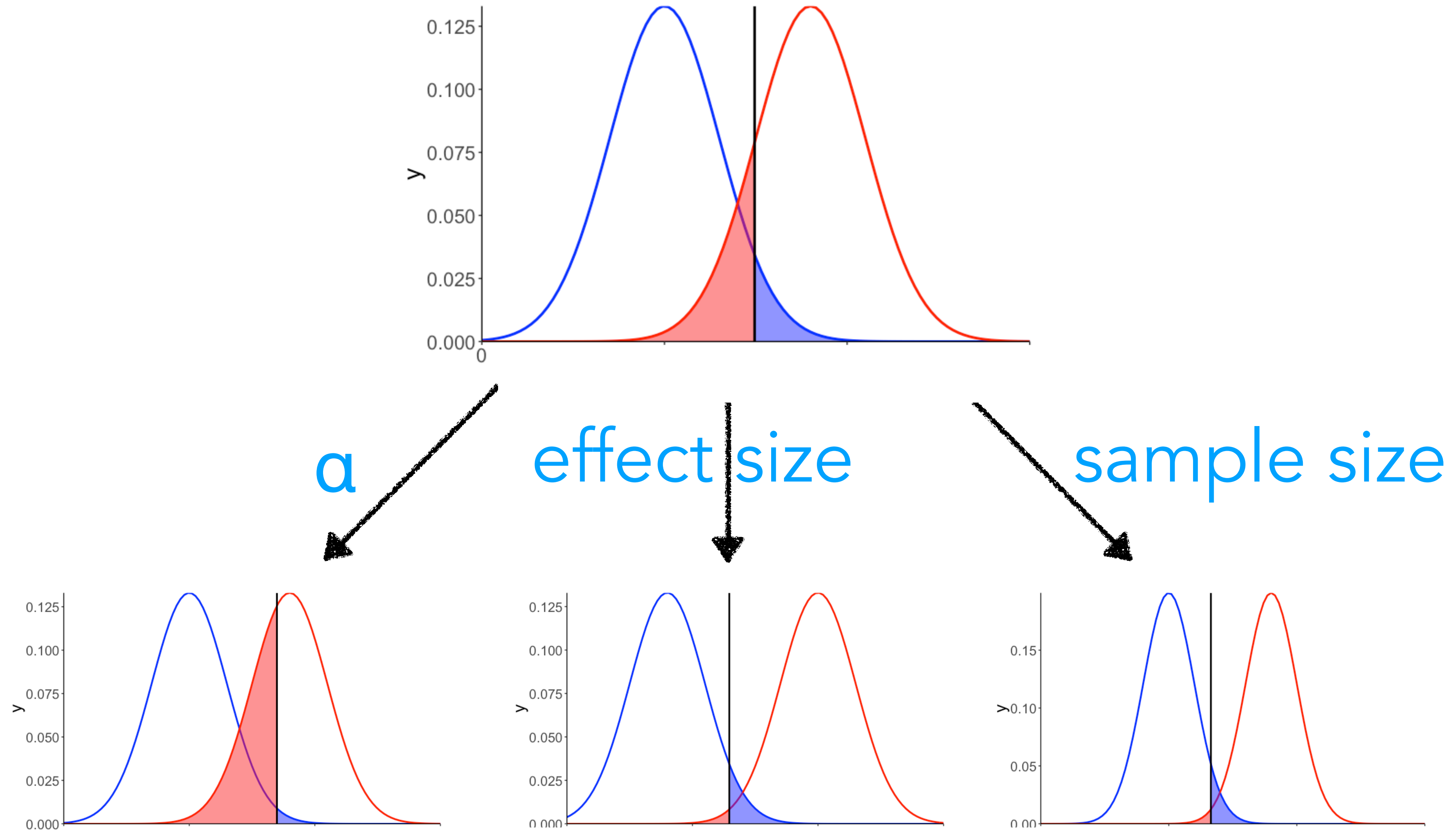
Power analysis

Calculating power

With great power comes ...



The knobs we can turn to affect power



Visualization demo

Settings

Solve for? ☐ Power ☐ Alpha ☐ n ☒ d

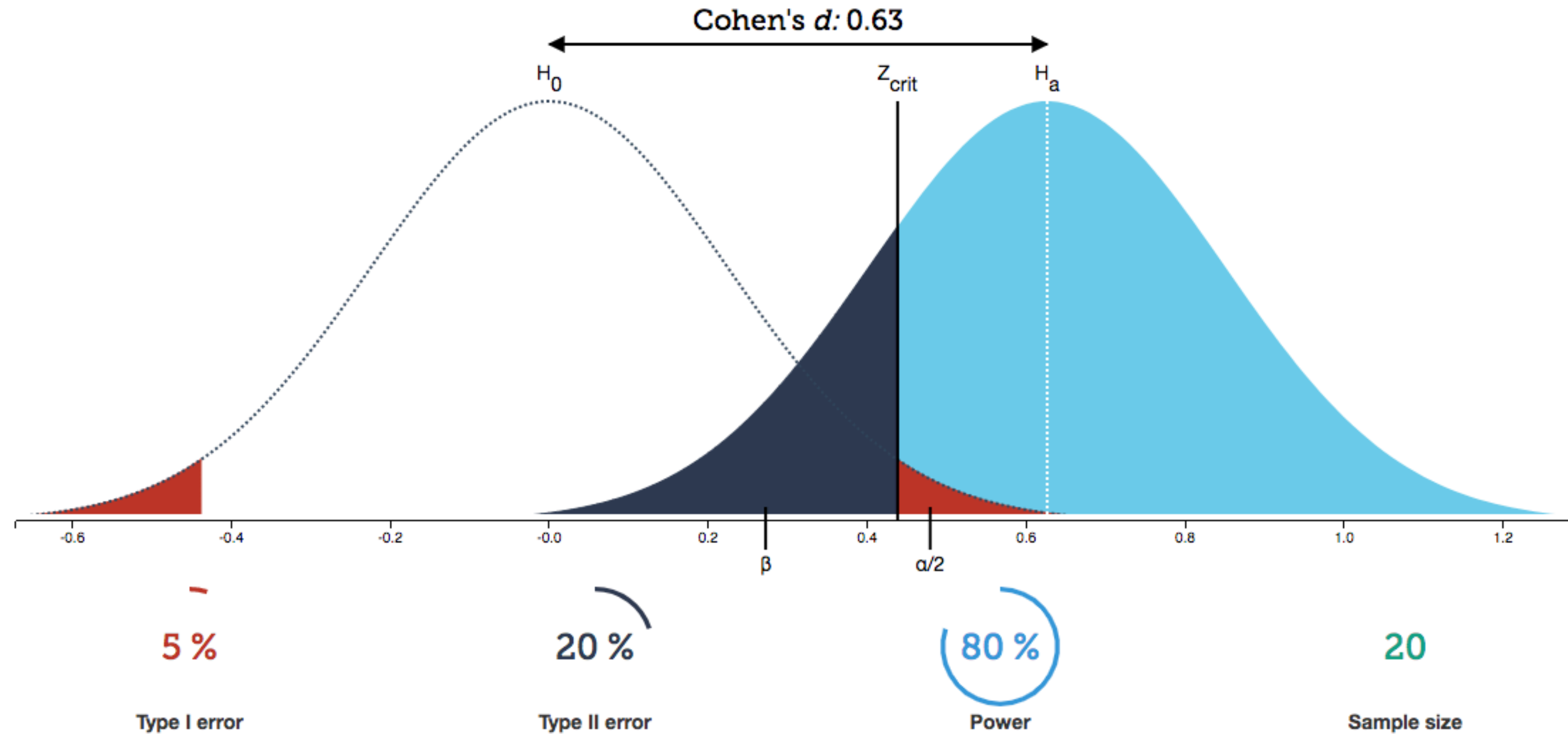
Power ($1-\beta = 0.8$)

Significance level ($\alpha = 0.05$)

Sample size ($n = 20$)

One-tailed Two-tailed

Reset zoom



<https://rpsychologist.com/d3/NHST/>

The **power** of a binary hypothesis test is the probability that the test rejects the null hypothesis (H_0) when a **specific** alternative hypothesis (H_1) is true.

H_0 : Students and non-students have the same balance.

Model C

$$Y_i = \beta_0 + \epsilon_i$$

$$\beta_1 = 0$$

H_1 : Students and non-students have different balances.

Model A

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

$$\beta_1 \neq 0$$

**We cannot calculate power in this case.
We need a specific alternative hypothesis!**

The **power** of a binary hypothesis test is the probability that the test rejects the null hypothesis (H_0) when a **specific** alternative hypothesis (H_1) is true.

H_0 : Students and non-students have the same balance.

Model C

$$Y_i = \beta_0 + \epsilon_i$$

$$\beta_1 = 0$$

H_1 : Students and non-students have different balances.

Model A

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

$$\beta_1 = 300$$

We can calculate power in this case (since we have a specific alternative hypothesis)!

Effect sizes

Effect sizes

- a p-value tells us whether we can reject the H_0
- effect sizes is a measure of the strength of the effect

Why can't we just use p-values
as a measure of the effect size?

$$F = \frac{\text{PRE}/(\text{PA} - \text{PC})}{(1 - \text{PRE})/(n - \text{PA})}$$

PRE = proportional reduction in error

PA = # parameters in the augmented model

PC = # parameters in the compact model

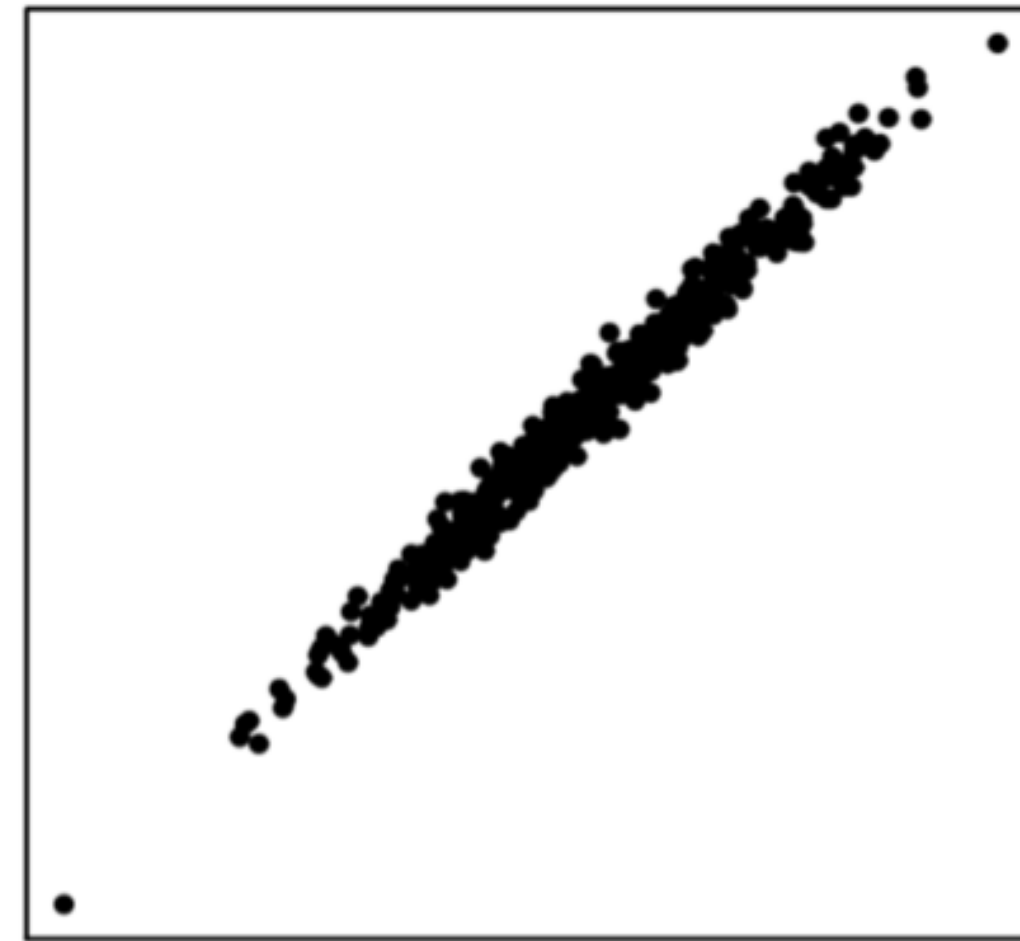
n = sample size

any PRE will become
significant if n gets
large enough

**statistical vs. practical
significance**

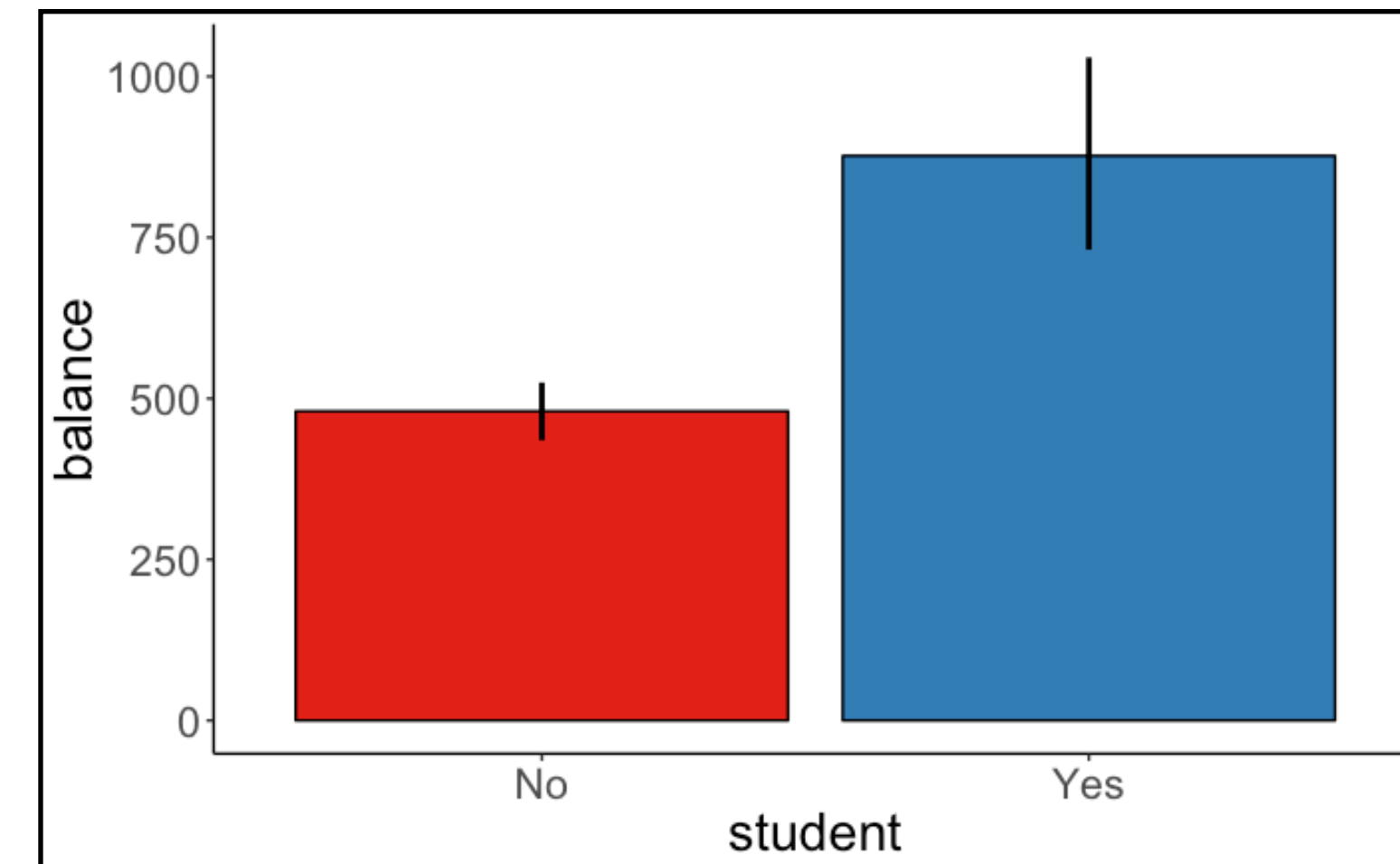
Common effect sizes

Relationships between variables



r correlation

Differences between groups



Cohen's d

Correlation

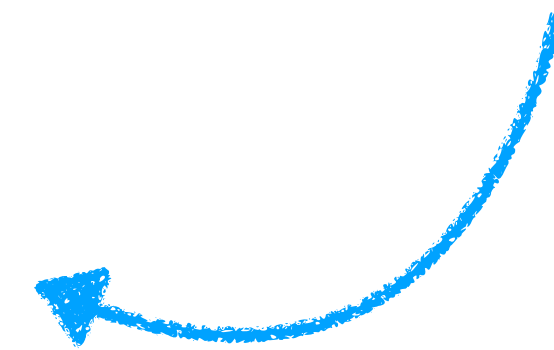
Pearson correlation

$$r(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \cdot \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}}$$

Cohen's guidelines for the social sciences

Effect size	r
Small	0.1
Medium	0.3
Large	0.5

depends very
much on the
domain



Cohen's d

- standardized difference between two means

absolute
difference
between means

$$d = \frac{|\bar{y}_1 - \bar{y}_2|}{s_p}$$

pooled standard variation

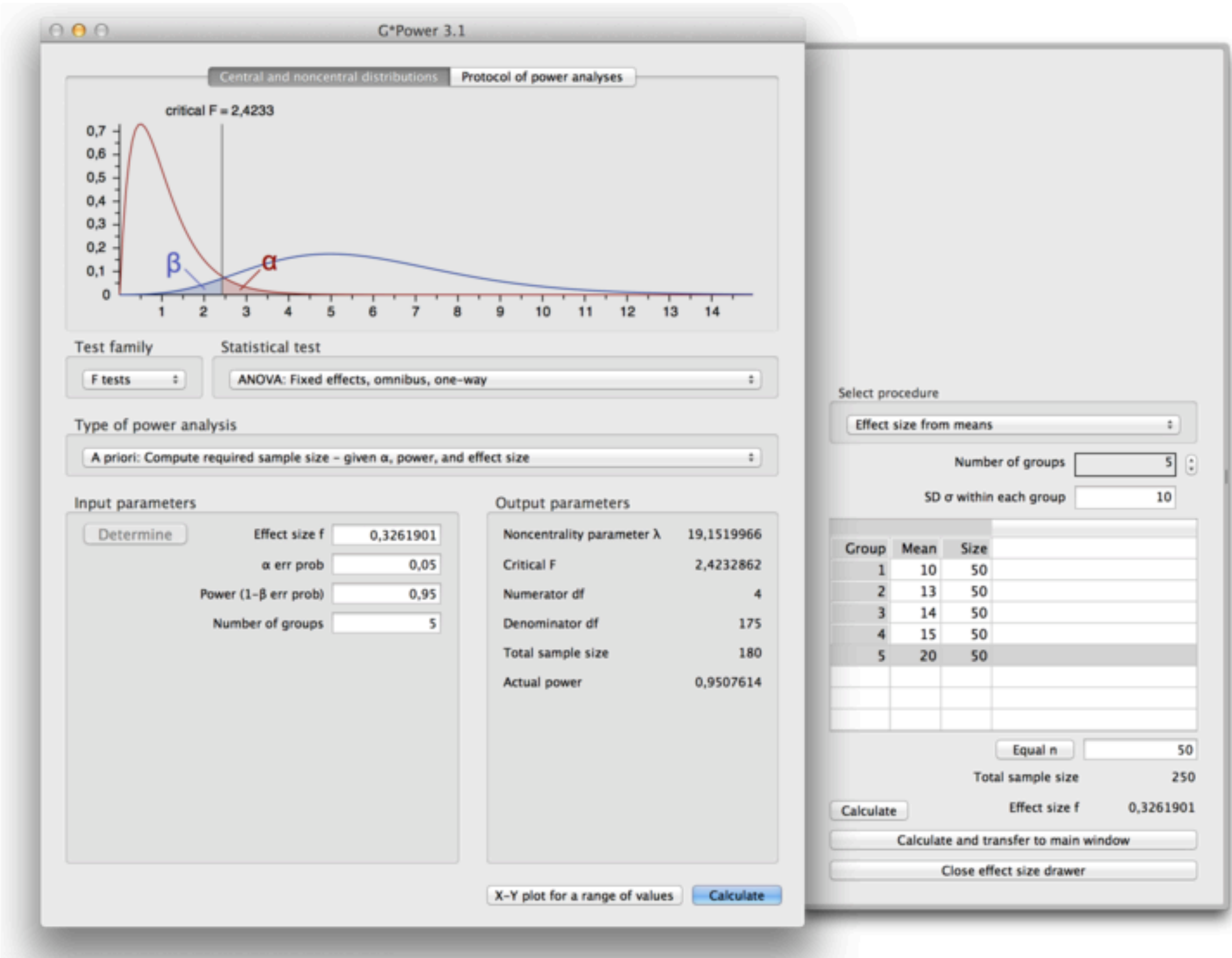
$$s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

<i>Effect size</i>	<i>d</i>
Very small	0.01
Small	0.20
Medium	0.50
Large	0.80
Very large	1.20
Huge	2.0

Difference between two means in pooled standard deviation

Determining sample size

How many participants do I need to run to have a good chance of detecting a true effect?

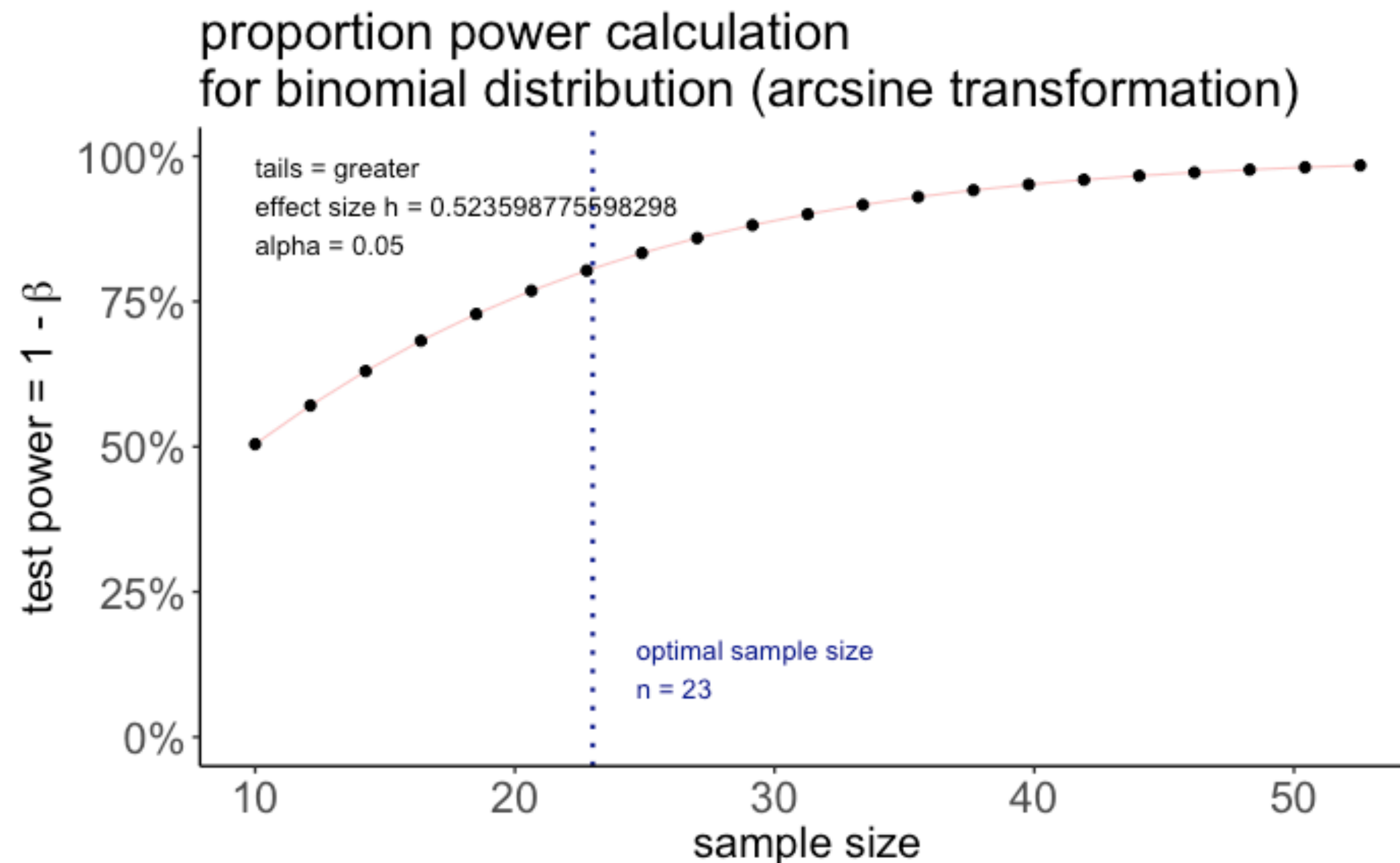


<http://www.gpower.hhu.de/>

"pwr" package in R

Option 2

```
1 library("pwr")
2 pwr.p.test(h = ES.h(p1 = 0.75, p2 = 0.50),
3           sig.level = 0.05,
4           power = 0.80,
5           alternative = "greater") %>%
6   plot()
```



Power simulation recipe

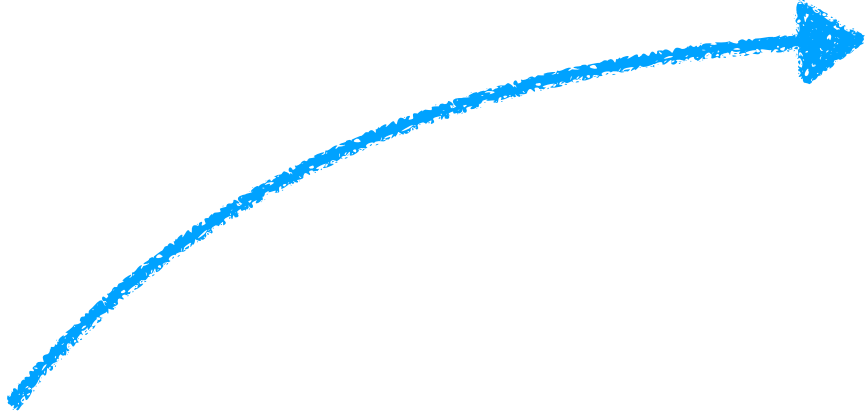
- assume:
 - α , n , effect size
- simulate a large number of data sets of size n with the specified effect size
- for each data set, run a statistical test to calculate the p-value
- determine the probability of rejecting the H_0 (given that H_1 is true)

Let's simulate ...

```
1 # make reproducible
2 set.seed(1)
3
4 # number of simulations
5 n_simulations = 5
6
7 # run simulation
8 expand_grid(n = seq(10, 40, 2),
9            simulation = 1:n_simulations,
10            p = 0.75) %>%
11   mutate(index = 1:n(),
12          .before = n) %>%
13   group_by(index, n, p, simulation) %>%
14   mutate(response = rbinom(n = 1,
15                            size = n,
16                            prob = p),
17          p.value = binom.test(x = response,
18                               n = n,
19                               p = 0.5,
20                               alternative = "two.sided")$p.value) %>%
21   group_by(n, p) %>%
22   summarize(power = sum(p.value < 0.05) / n())
```

Let's simulate ...

```
1 # make reproducible
2 set.seed(1)
3
4 # number of simulations
5 n_simulations = 5
6
7 # run simulation
8 expand_grid(n = seq(10, 40, 2),
9            simulation = 1:n_simulations,
10            p = 0.75) %>%
11   mutate(index = 1:n(),
12          .before = n) %>%
```



index	n	simulation	p
1	10	1	0.75
2	10	2	0.75
3	10	3	0.75
4	10	4	0.75
5	10	5	0.75
6	12	1	0.75
7	12	2	0.75
8	12	3	0.75
9	12	4	0.75
10	12	5	0.75

Let's simulate ...

```
1 # make reproducible
2 set.seed(1)
3
4 # number of simulations
5 n_simulations = 5
6
7 # run simulation
8 expand_grid(n = seq(10, 40, 2),
9             simulation = 1:n_simulations,
10             p = 0.75) %>%
11   mutate(index = 1:n(),
12          .before = n) %>%
13   group_by(index, n, p, simulation) %>%
14   mutate(response = rbinom(n = 1,
15                             size = n,
16                             prob = p),
```

index	n	simulation	p	response
1	10	1	0.75	8
2	10	2	0.75	7
3	10	3	0.75	8
4	10	4	0.75	8
5	10	5	0.75	7
6	12	1	0.75	10
7	12	2	0.75	8
8	12	3	0.75	11
9	12	4	0.75	10
10	12	5	0.75	11



Let's simulate ...

```
1 # make reproducible
2 set.seed(1)
3
4 # number of simulations
5 n_simulations = 5
6
7 # run simulation
8 expand_grid(n = seq(10, 40, 2),
9            simulation = 1:n_simulations,
10            p = 0.75) %>%
11   mutate(index = 1:n(),
12          .before = n) %>%
13   group_by(index, n, p, simulation) %>%
14   mutate(response = rbinom(n = 1,
15                            size = n,
16                            prob = p),
17          p.value = binom.test(x = response,
18                               n = n,
19                               p = 0.5,
20                               alternative = "two.sided")$p.value) %>%
```

index	n	simulation	p	response	p.value
1	10	1	0.75	8	0.11
2	10	2	0.75	7	0.34
3	10	3	0.75	8	0.11
4	10	4	0.75	8	0.11
5	10	5	0.75	7	0.34
6	12	1	0.75	10	0.04
7	12	2	0.75	8	0.39

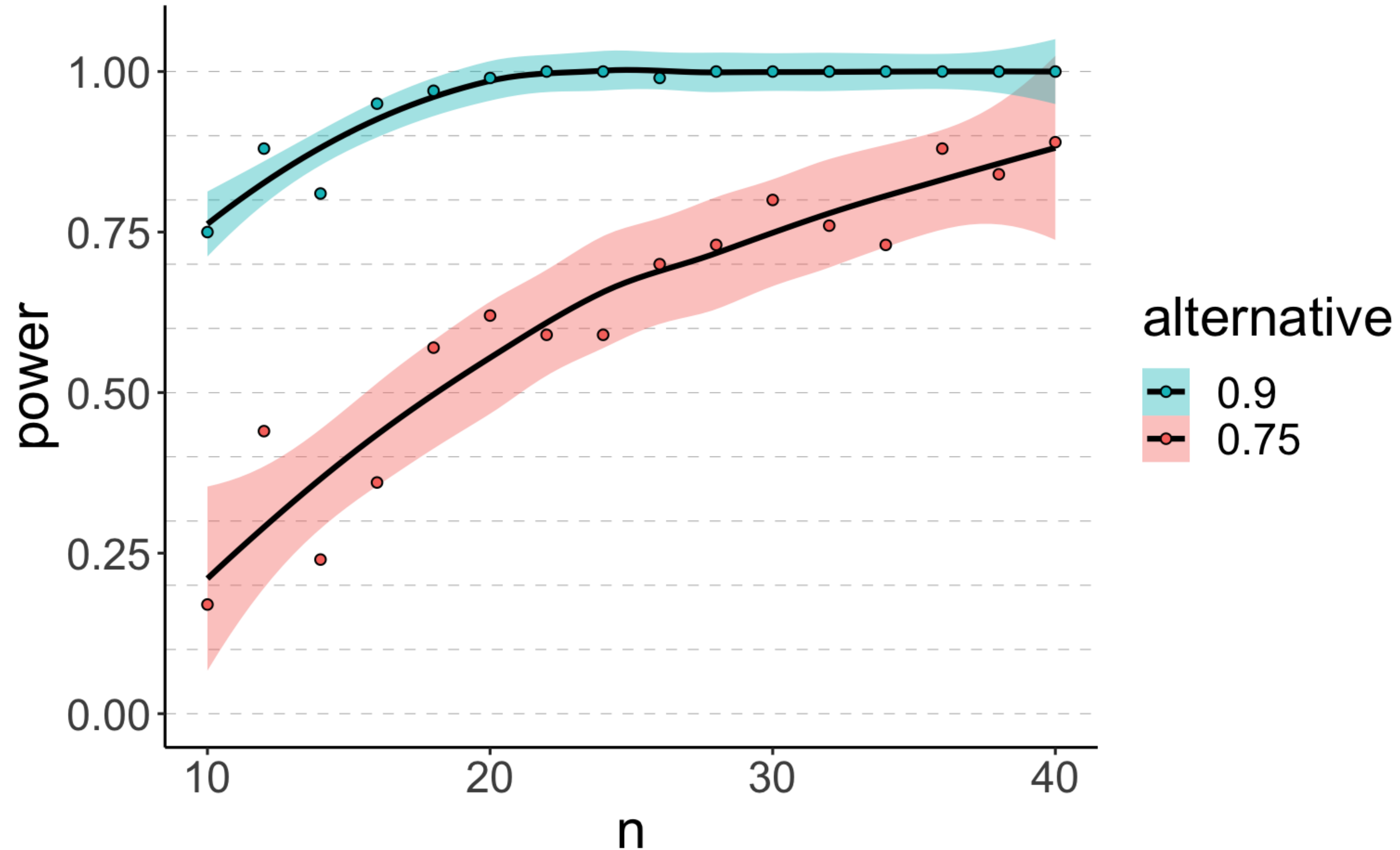
Let's simulate ...

```
1 # make reproducible
2 set.seed(1)
3
4 # number of simulations
5 n_simulations = 5
6
7 # run simulation
8 expand_grid(n = seq(10, 40, 2),
9             simulation = 1:n_simulations,
10             p = 0.75) %>%
11   mutate(index = 1:n(),
12          .before = n) %>%
13   group_by(index, n, p, simulation) %>%
14   mutate(response = rbinom(n = 1,
15                             size = n,
16                             prob = p),
17          p.value = binom.test(x = response,
18                               n = n,
19                               p = 0.5,
20                               alternative = "two.sided")$p.value) %>%
21   group_by(n, p) %>%
22   summarize(power = sum(p.value < 0.05) / n())
```

n	p	power
10	0.75	0.2
12	0.75	0.2
14	0.75	0.4
16	0.75	0.2
18	0.75	0.6
20	0.75	0.8
22	0.75	0.6
24	0.75	0.4
26	0.75	0.6
28	0.75	0.8
30	0.75	0.8

Let's simulate ...

in this example, I looked at the
power for two different
alternative hypotheses



Let's simulate ...

- here, I've used a simple example (Binomial test)
- but: we can use the same recipe for any statistical test that we are planning on running

Power simulation recipe

- assume:
 - α , n , effect size
- simulate a large number of data sets of size n with the specified effect size
- for each data set, run a statistical test to calculate the p-value for a given α
- determine the probability of rejecting the H_0 (given that H_1 is true)

INTERACTIVE COURSE

Foundations of Functional Programming with purrr

Start Course For Free

Bookmark

🕒 4 hours

▶ 13 Videos

↔ 44 Exercises

👤 7,769 Participants

🏆 3,750 XP

Course Description

Lists can be difficult to both understand and manipulate, but they can pack a ton of information and are very powerful. In this course, you will learn to easily extract, summarize, and manipulate lists and how to export the data to your desired object, be it another list, a vector, or even something else! Throughout the course, you will work with the purrr package and a variety of datasets from the repurrrsive package, including data from Star Wars and Wes Anderson films and data collected about GitHub users and GitHub repos. Following this course, your list skills will be purrrfect!

1

Simplifying Iteration and Lists With purrr

FREE

0%

Iteration is a powerful way to make the computer do the work for you. It can also be an area of coding where it is easy to make lots of typos and simple mistakes. The purrr package helps simplify iteration so you can focus on the next step, instead of finding typos.

▶

The power of iteration

50 xp

↔

Introduction to iteration

100 xp

↔

Iteration with purrr

100 xp

↔

More iteration with for loops

100 xp

↔

More iteration with purrr

100 xp

▶

Subsetting lists; it's not that hard!

50 xp

↔

Subsetting lists

100 xp

↔

Subsetting list elements

100 xp

DataCamp Content Creator

Course Instructor

This instructor prefers to remain anonymous.

See More

COLLABORATOR(S)

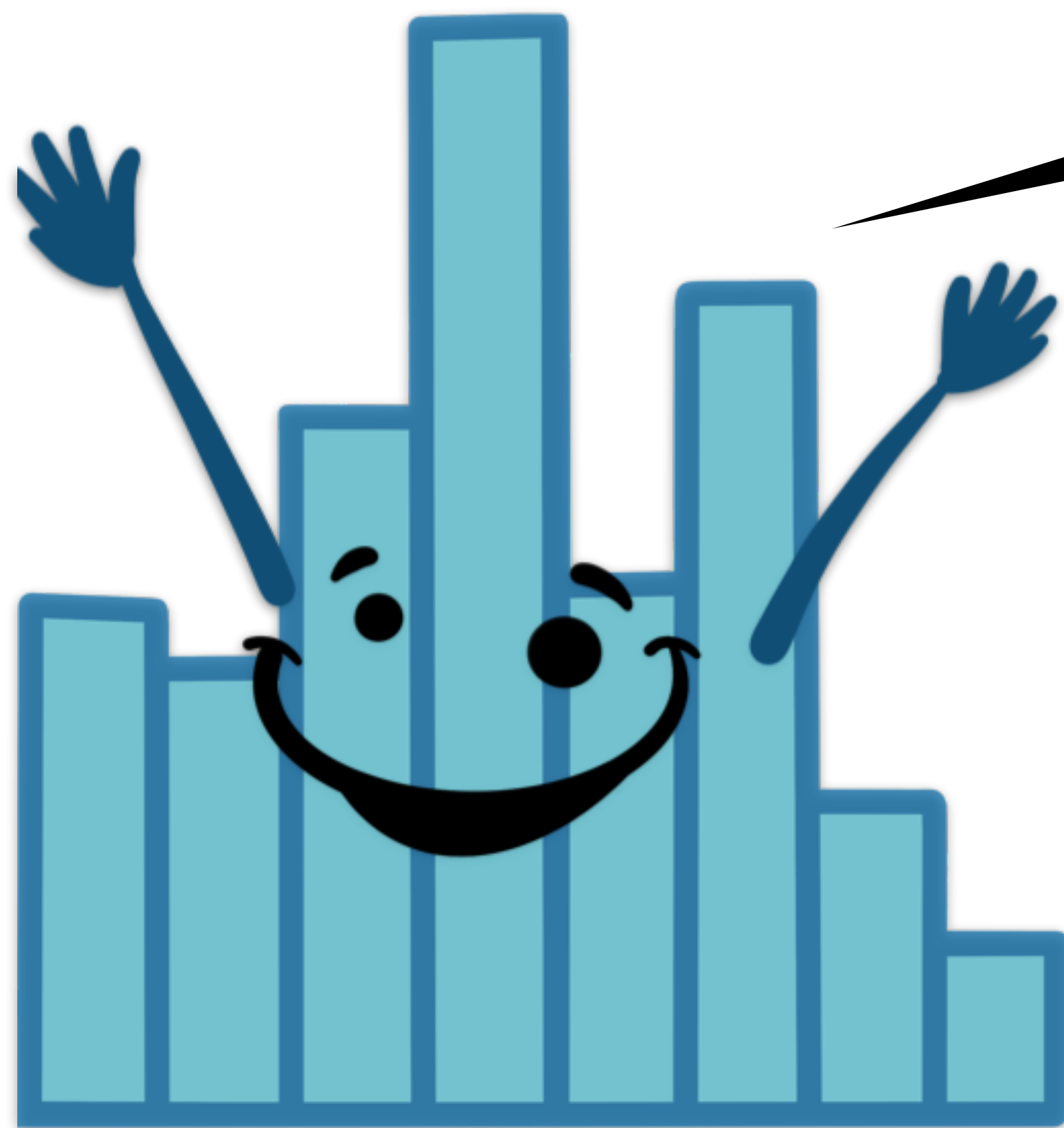
Chester Ismay

Becca Robins

<https://www.datacamp.com/courses/foundations-of-functional-programming-with-purrr>

41

02:00
stretch break!



Model comparison

The general procedure

1. Define H_0 as Model C (compact) and H_1 as Model A (augmented)
2. Fit model parameters to the data
3. Calculate the proportional reduction of error (PRE) in our sample
4. Decide whether the augmented model is **worth it** by comparing the observed PRE in our sample to the sampling distribution of PRE (assuming that H_0 is true)

Any problems with our approach?

sometimes it doesn't work ...

Model C

$$\text{balance}_i = \beta_0 + \epsilon_i$$

$$\text{balance}_i = \beta_0 + \beta_1 \cdot \text{student}_i + \epsilon_i$$

$$\text{balance}_i = \beta_0 + \beta_1 \cdot \text{student}_i + \epsilon_i$$

$$\text{balance}_i = \beta_0 + \beta_1 \cdot \text{student}_i + \epsilon_i$$

Model A

$$\text{balance}_i = \beta_0 + \beta_1 \cdot \text{student}_i + \beta_2 \cdot \text{age}_i + \epsilon_i$$



$$\text{balance}_i = \beta_0 + \beta_1 \cdot \text{student}_i + \beta_2 \cdot \text{age}_i + \epsilon_i$$



$$\text{balance}_i = \beta_0 + \beta_1 \cdot \text{age}_i + \epsilon_i$$



$$\text{balance}_i = \beta_0 + \beta_1 \cdot \text{age}_i + \beta_2 \cdot \text{degree}_i + \epsilon_i$$

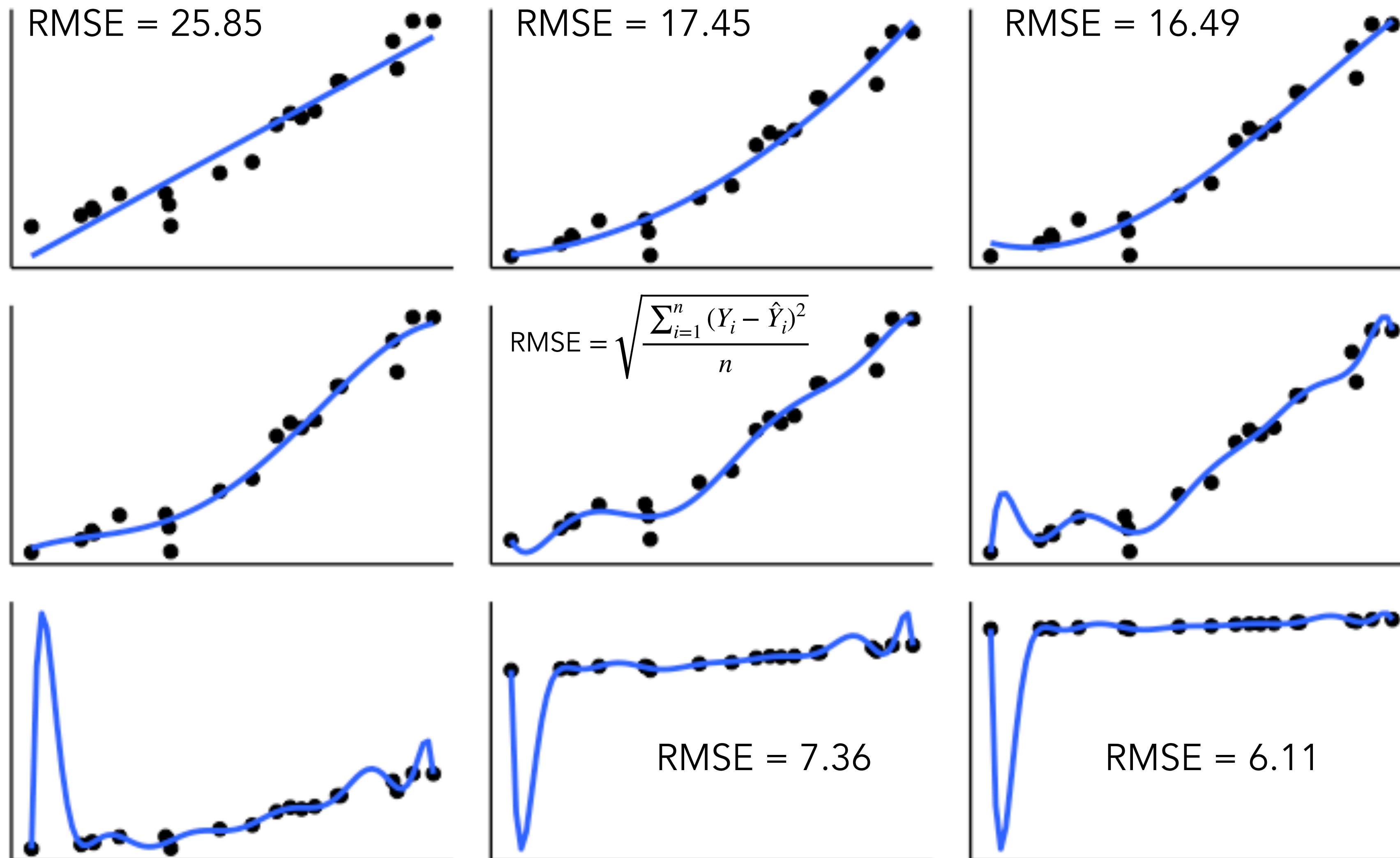


Tools for model comparison

- **anova** () : compare a compact model with an augmented model via the F-test (or likelihood ratio test)
 - problem: only works for **nested models** (where the augmented model contains all the predictors of the compact model and more)
- **What if we want to compare models that are not nested?**
 - 1 . Cross-validation
 - 2 . AIC and BIC
 - 3 . Bayesian data analysis (we'll get there soon)

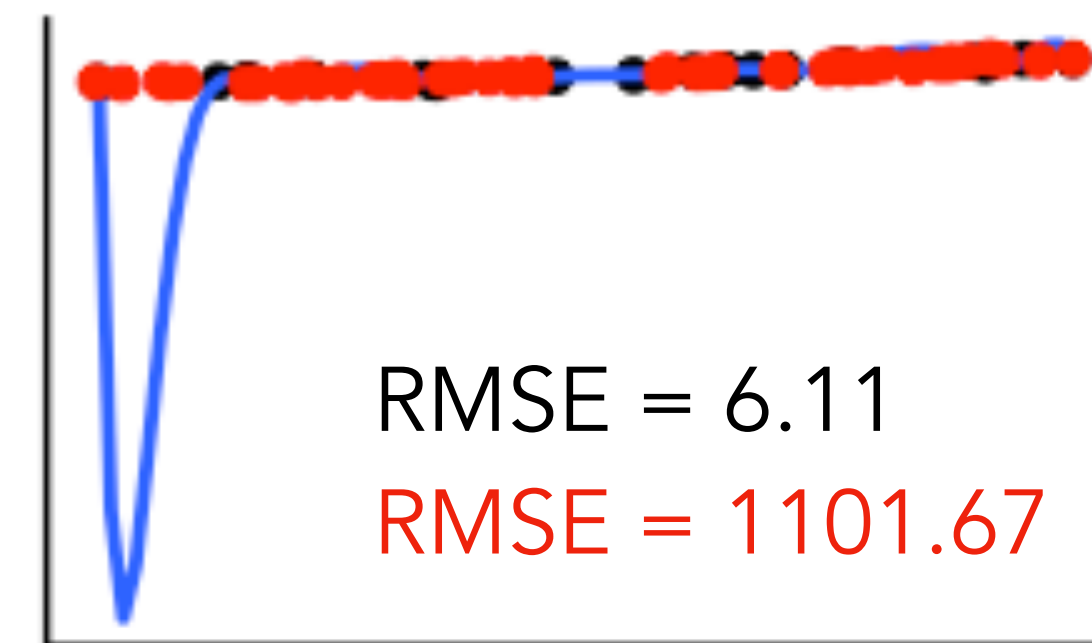
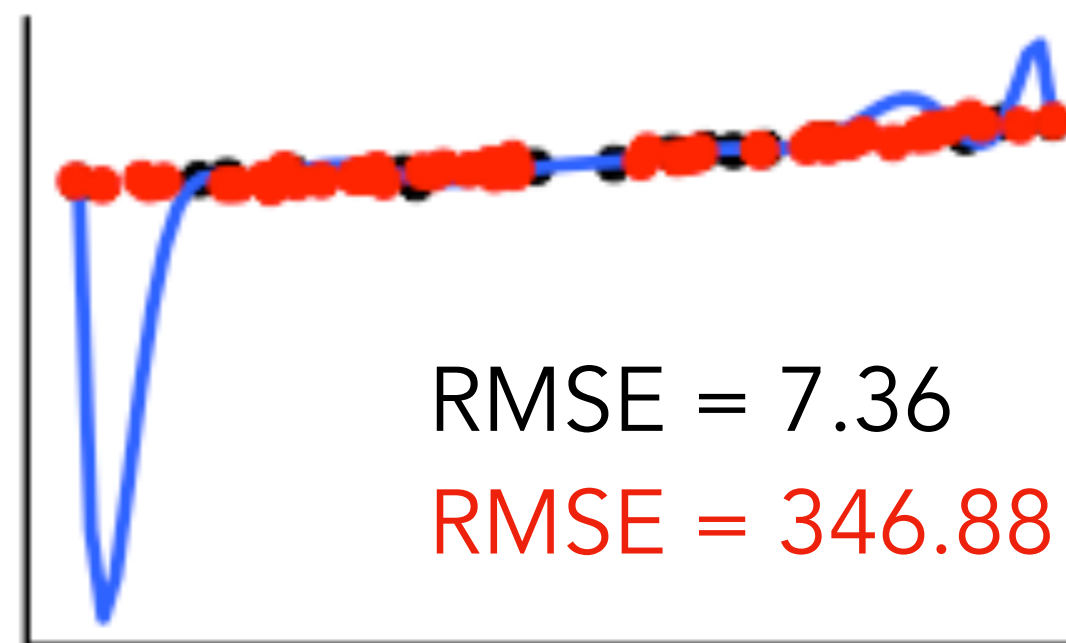
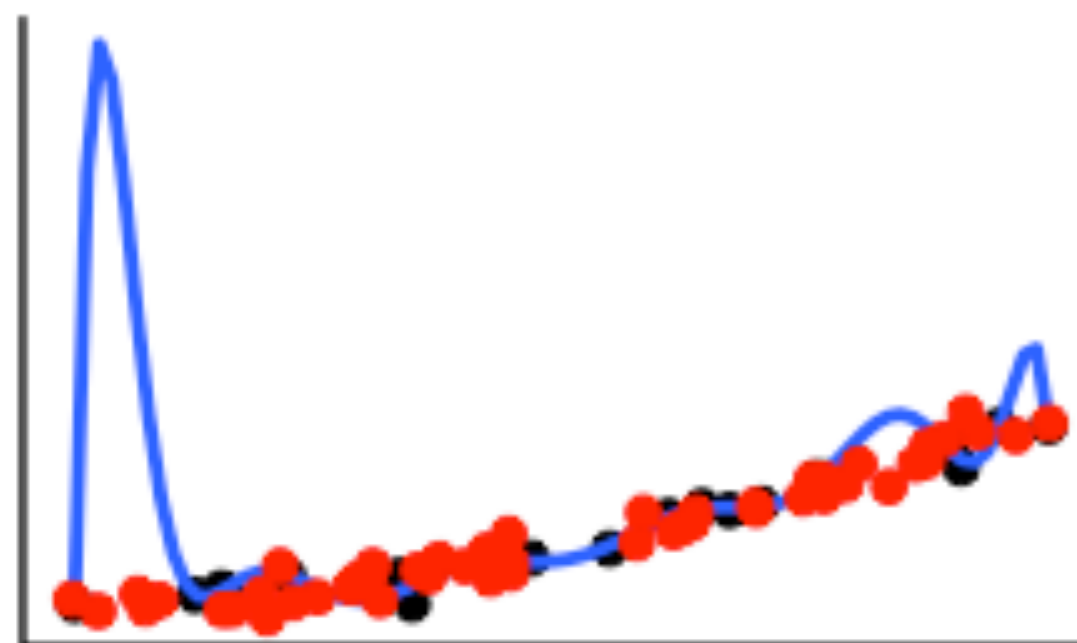
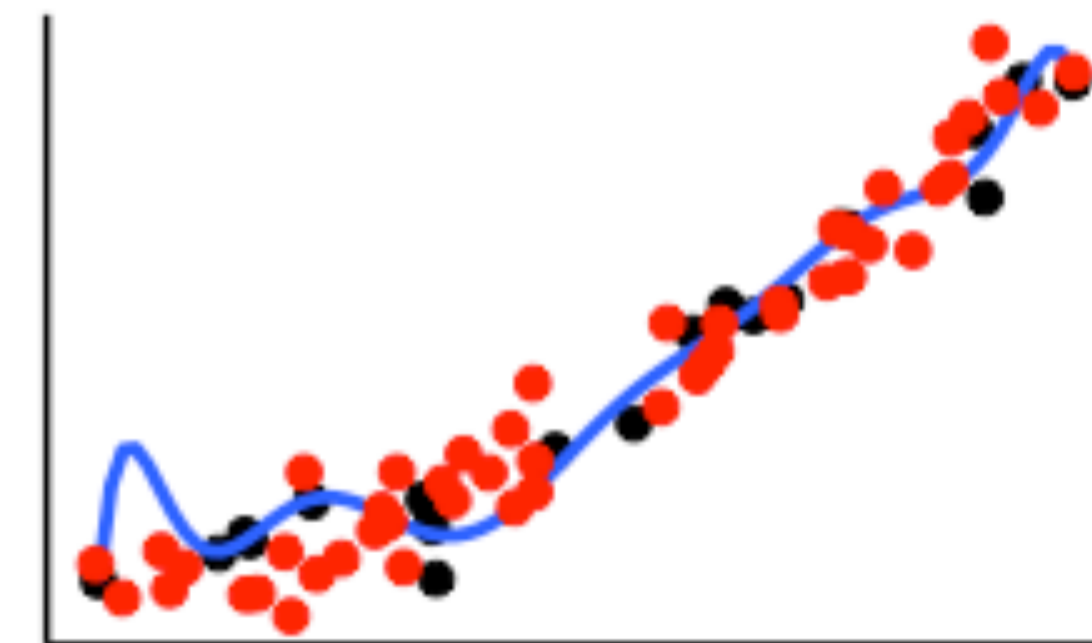
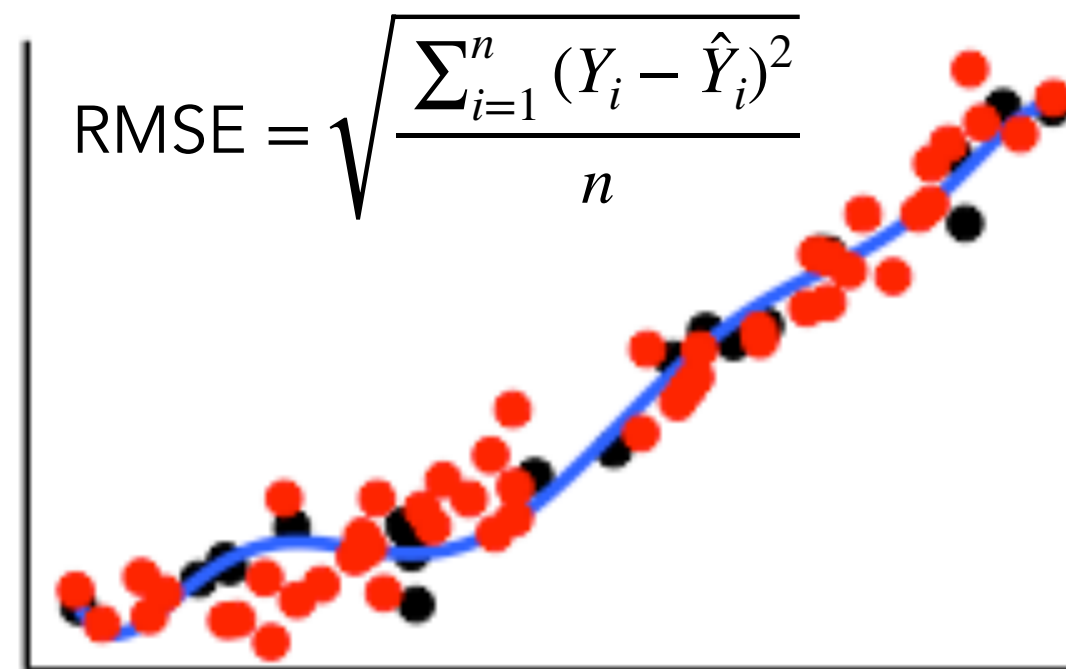
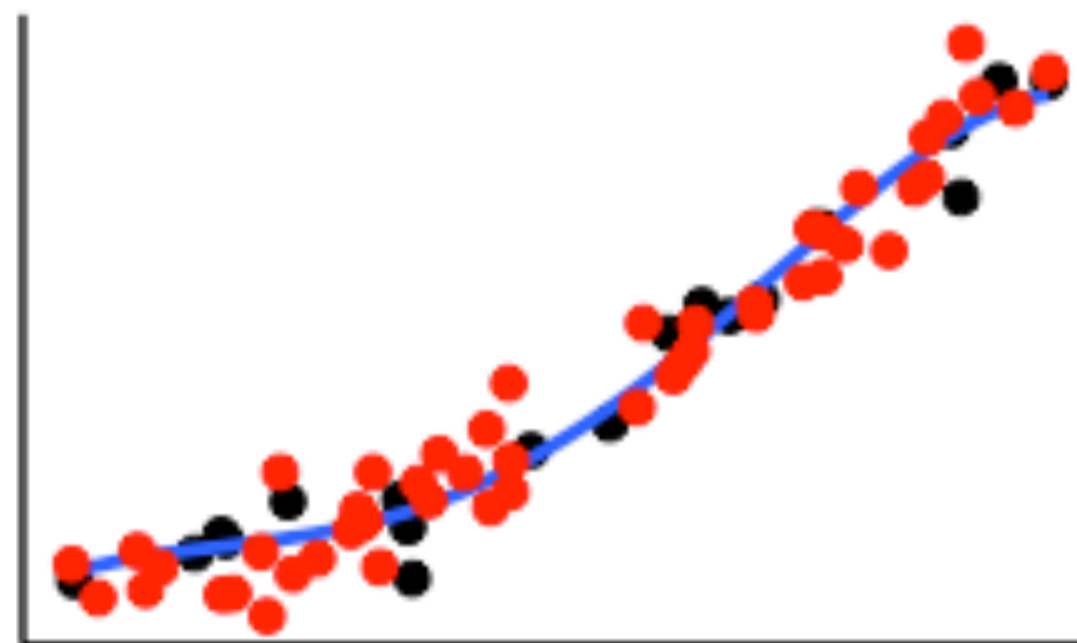
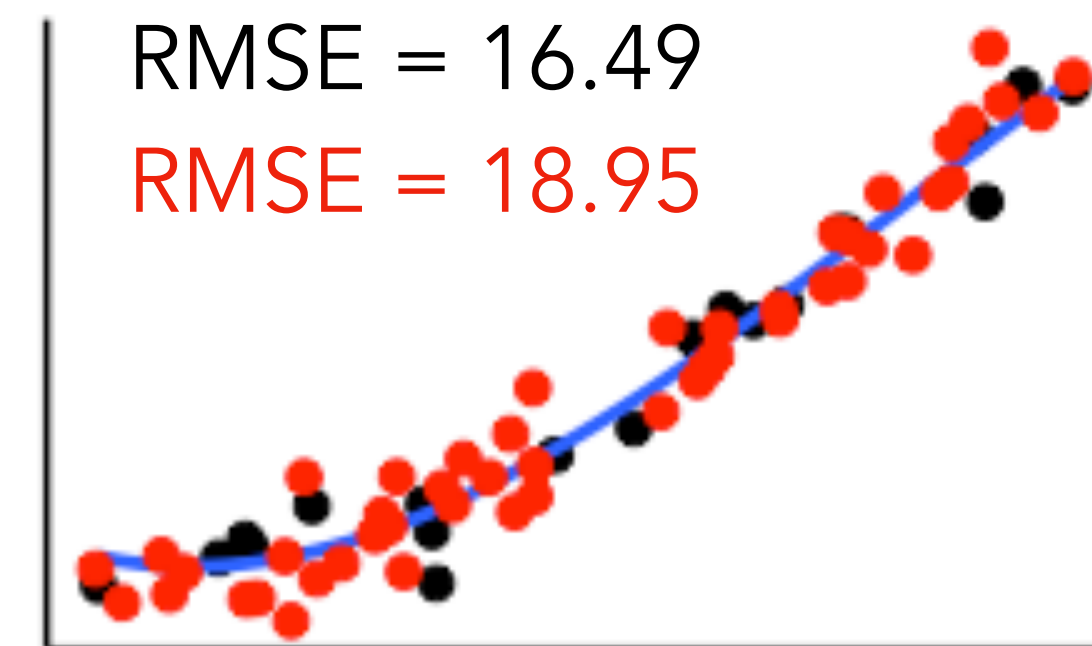
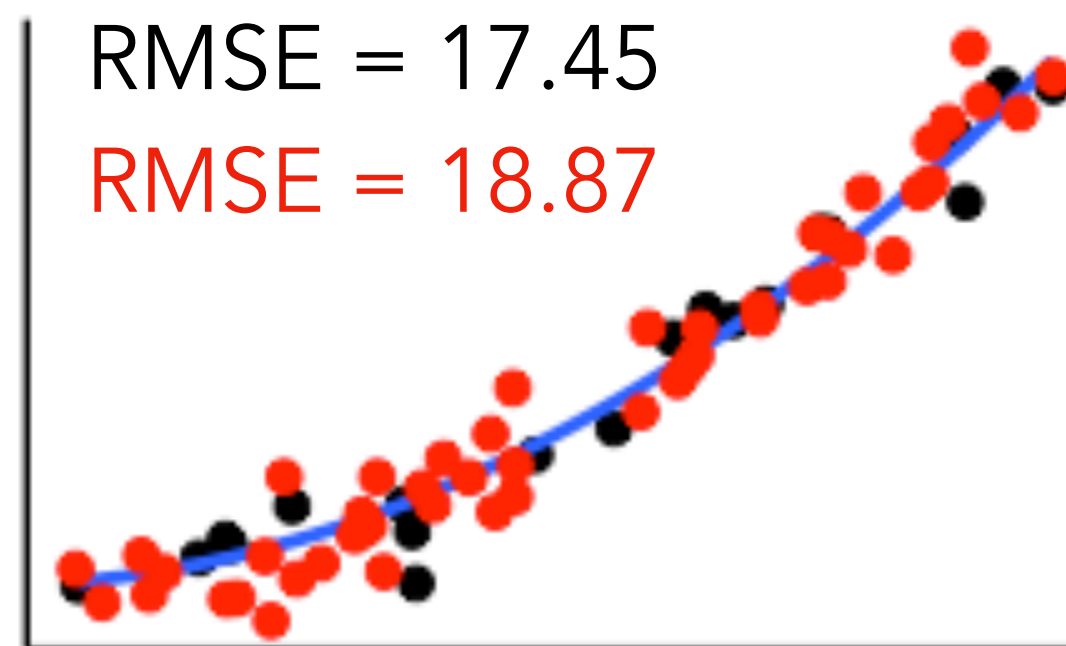
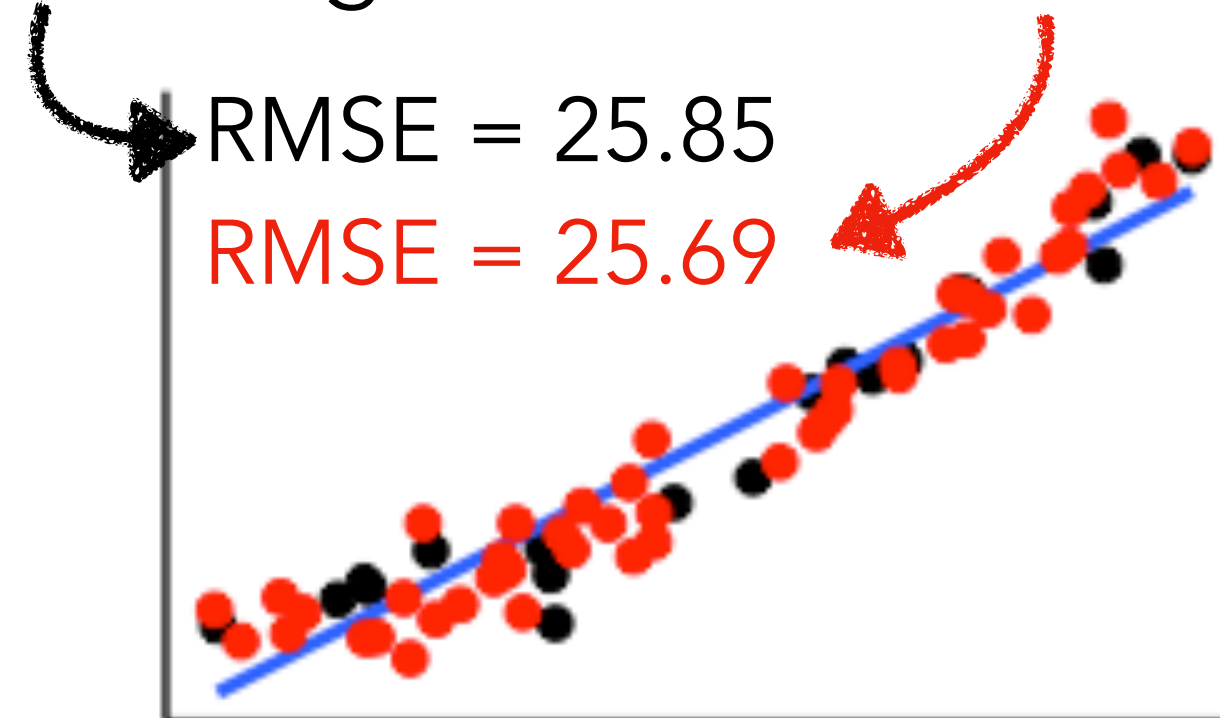
Cross-validation

Which model describes the data best?

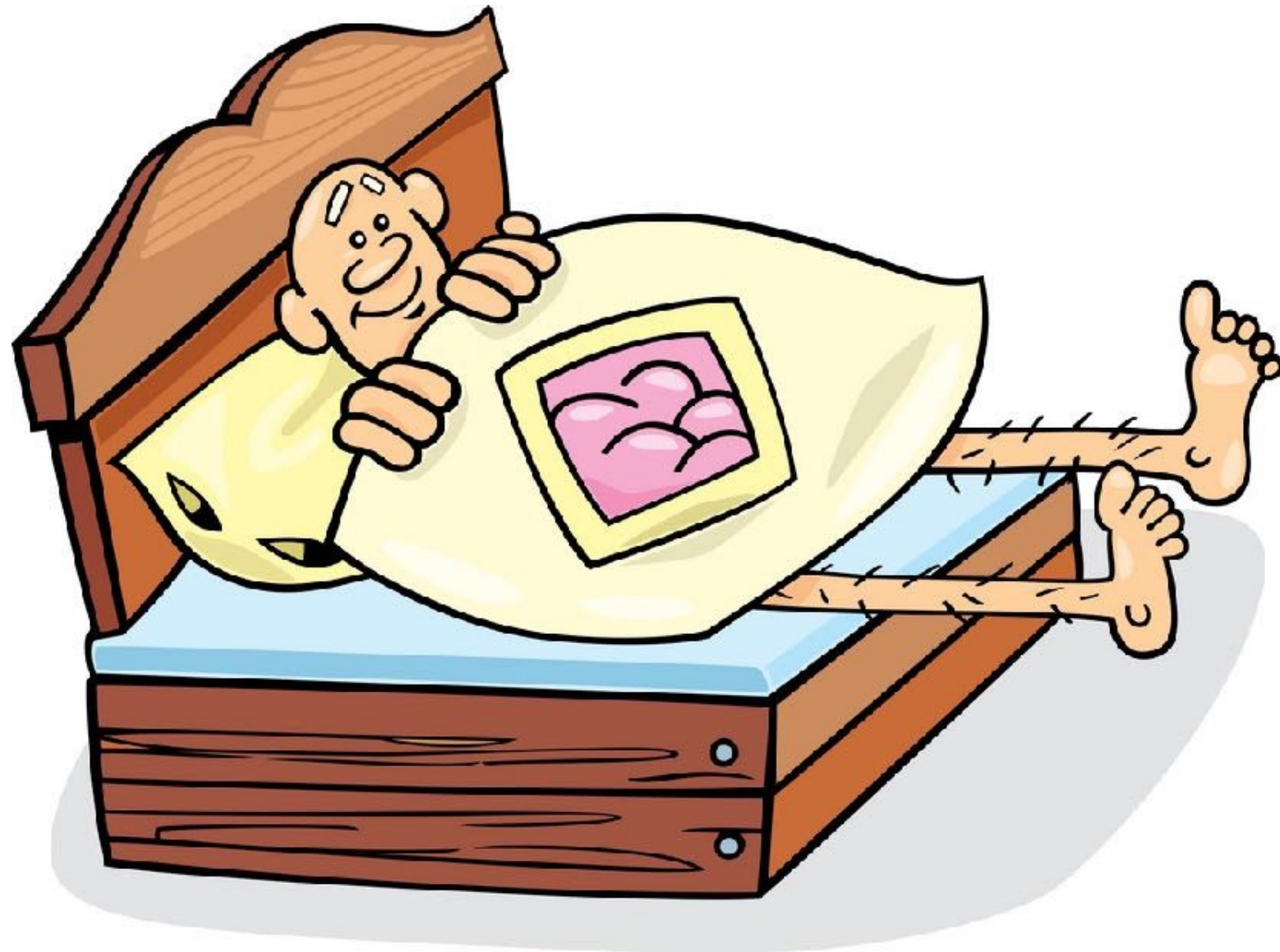


Which model describes the data best?

training error validation error



Underfitting vs. Overfitting



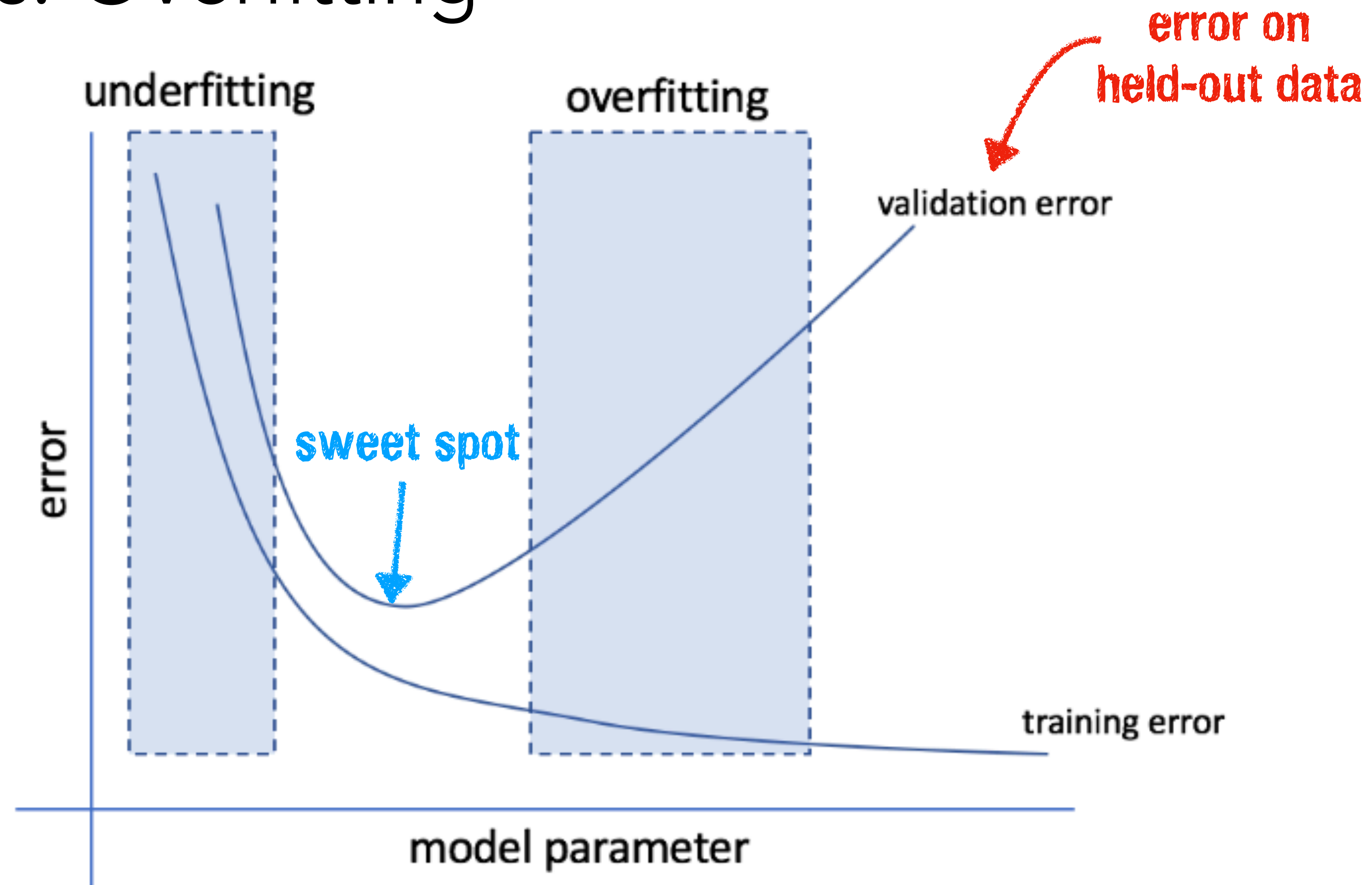
**ONE WAY TO EXPLAIN
UNDERFITTING**



Underfitting vs. Overfitting

- **a good model should:**
 - explain the actual data well
 - predict future data well
- **bias-variance tradeoff:**
 - **bias** = error from erroneous assumptions in the model, high bias can cause a model to miss the relevant relations between predictors and outcome **underfitting**
 - **variance** = error from sensitivity to small fluctuations in the data, high variance can cause a model to fit the random **noise** in the data **overfitting**

Underfitting vs. Overfitting

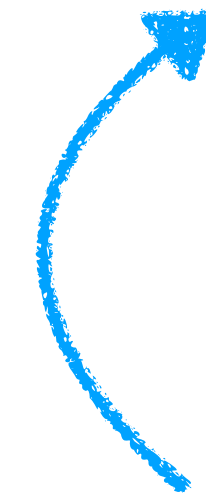


the goal is to find the **sweet spot** between underfitting and overfitting

Leave-one-out crossvalidation



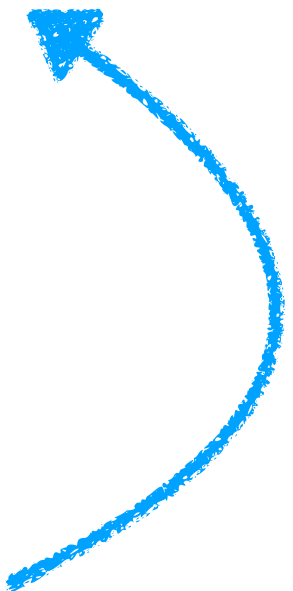
LOO



non-inspirational quote

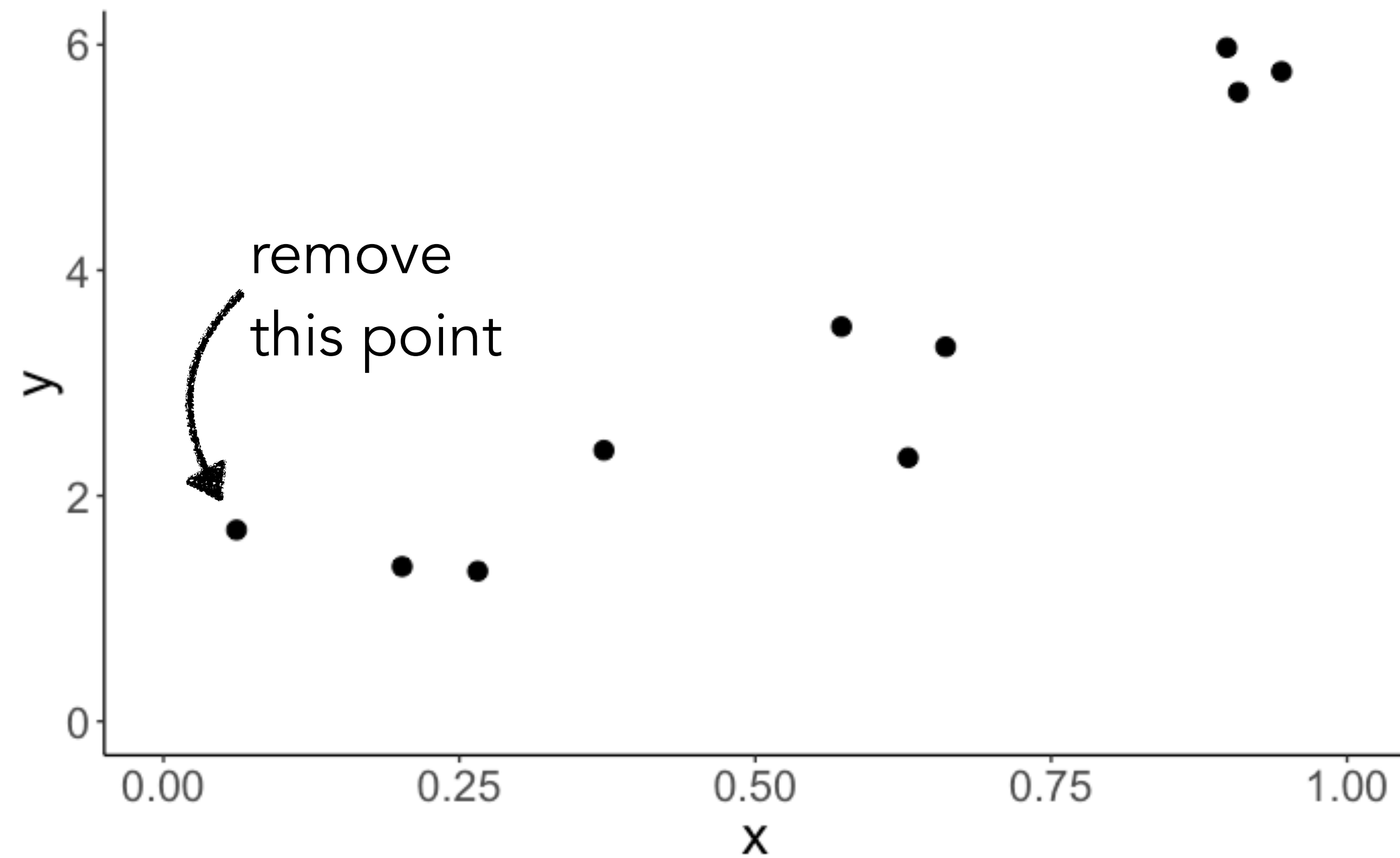
Leave one out cross-validation

- train the model on all the data points except for one
- calculate the prediction error for the held-out data point

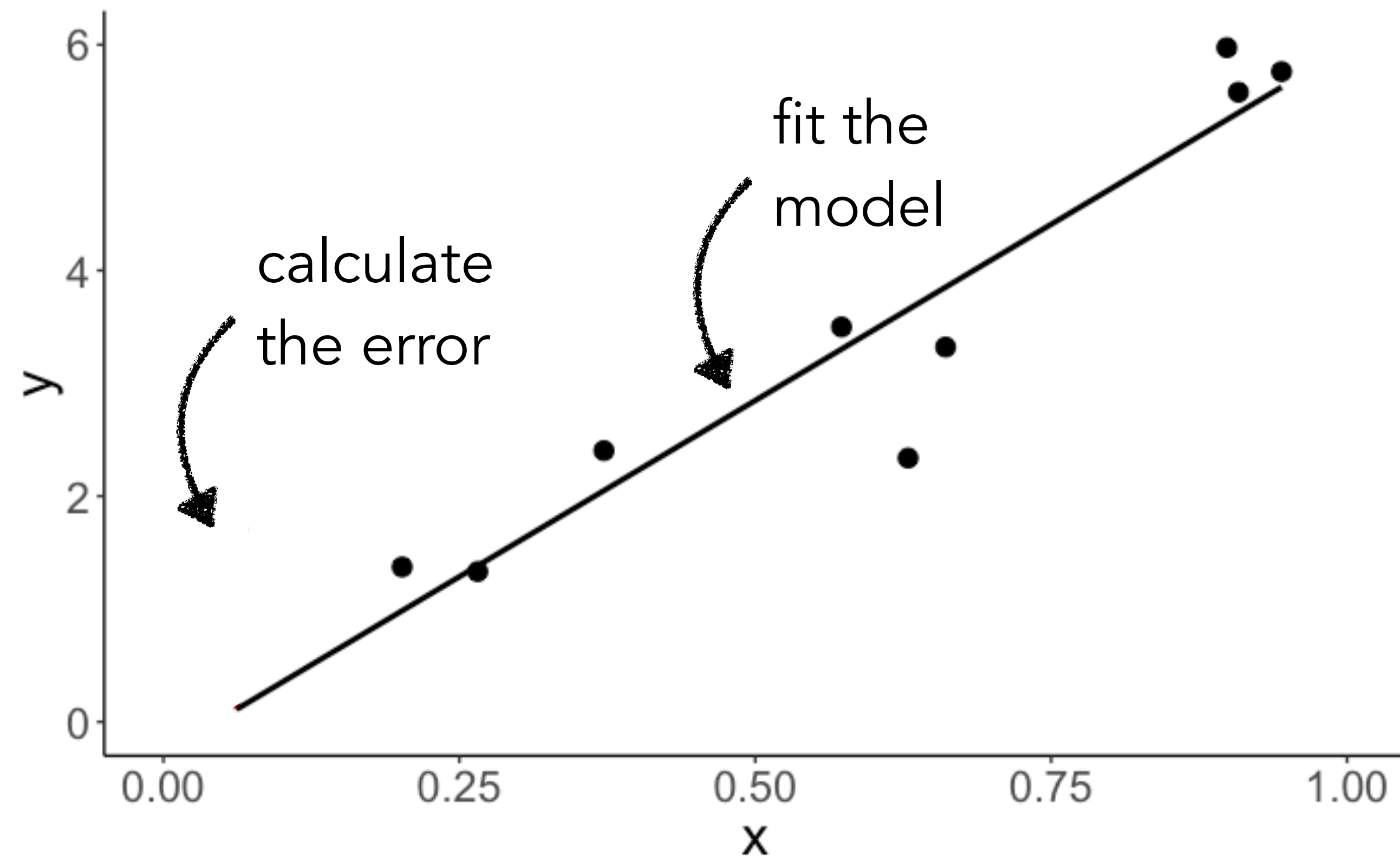


**repeat for all
data points**

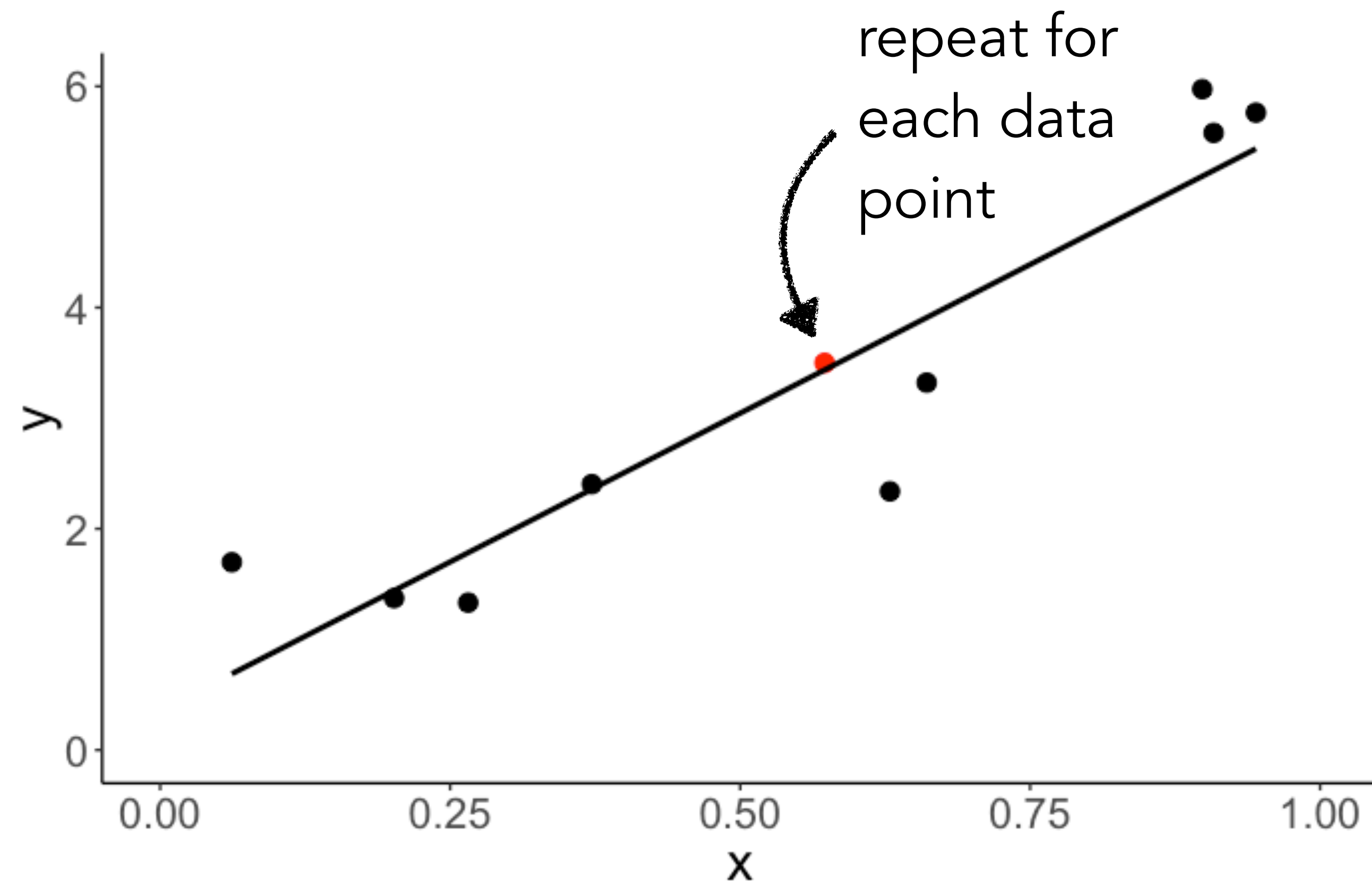
Leave one out cross-validation



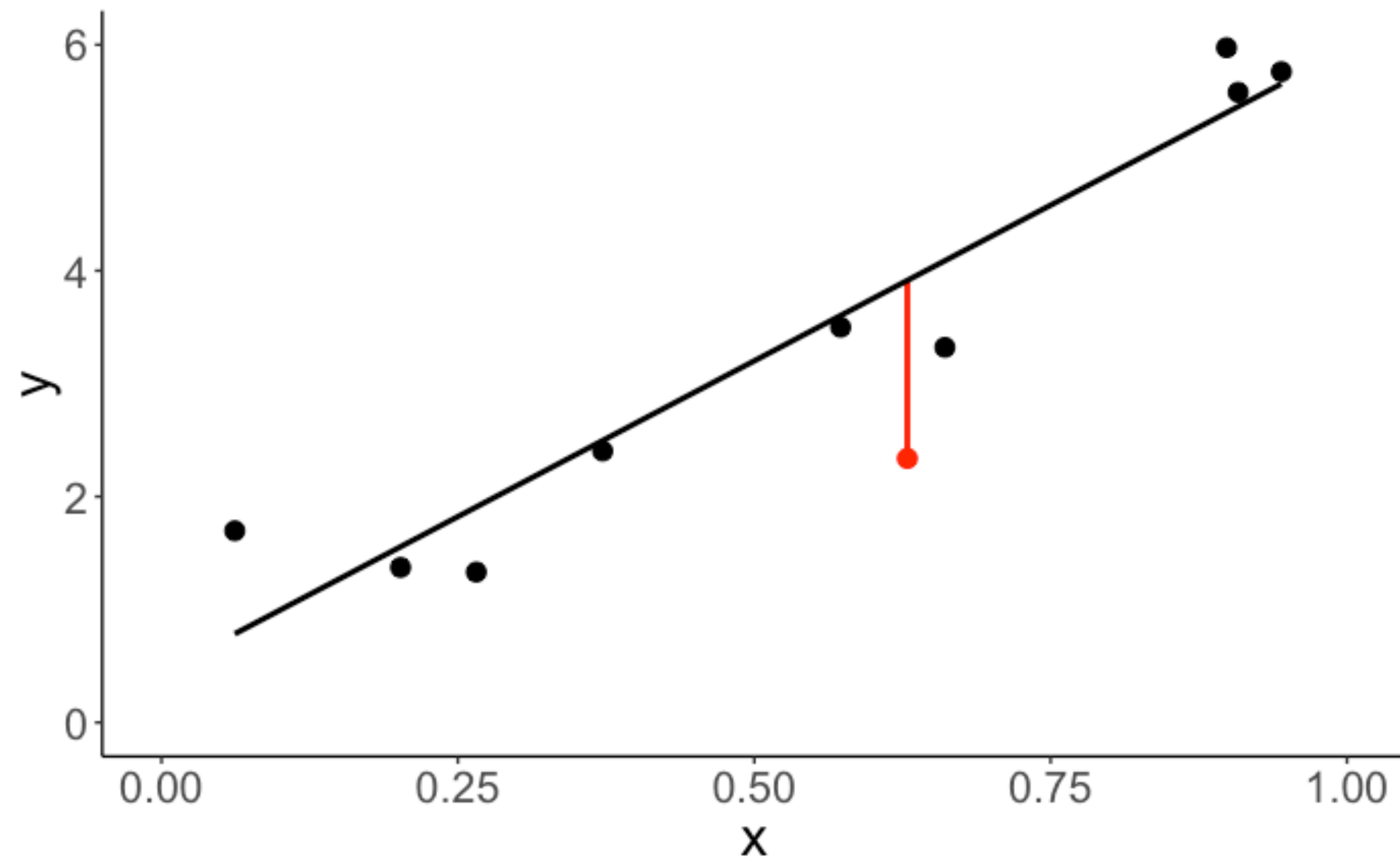
Leave one out cross-validation



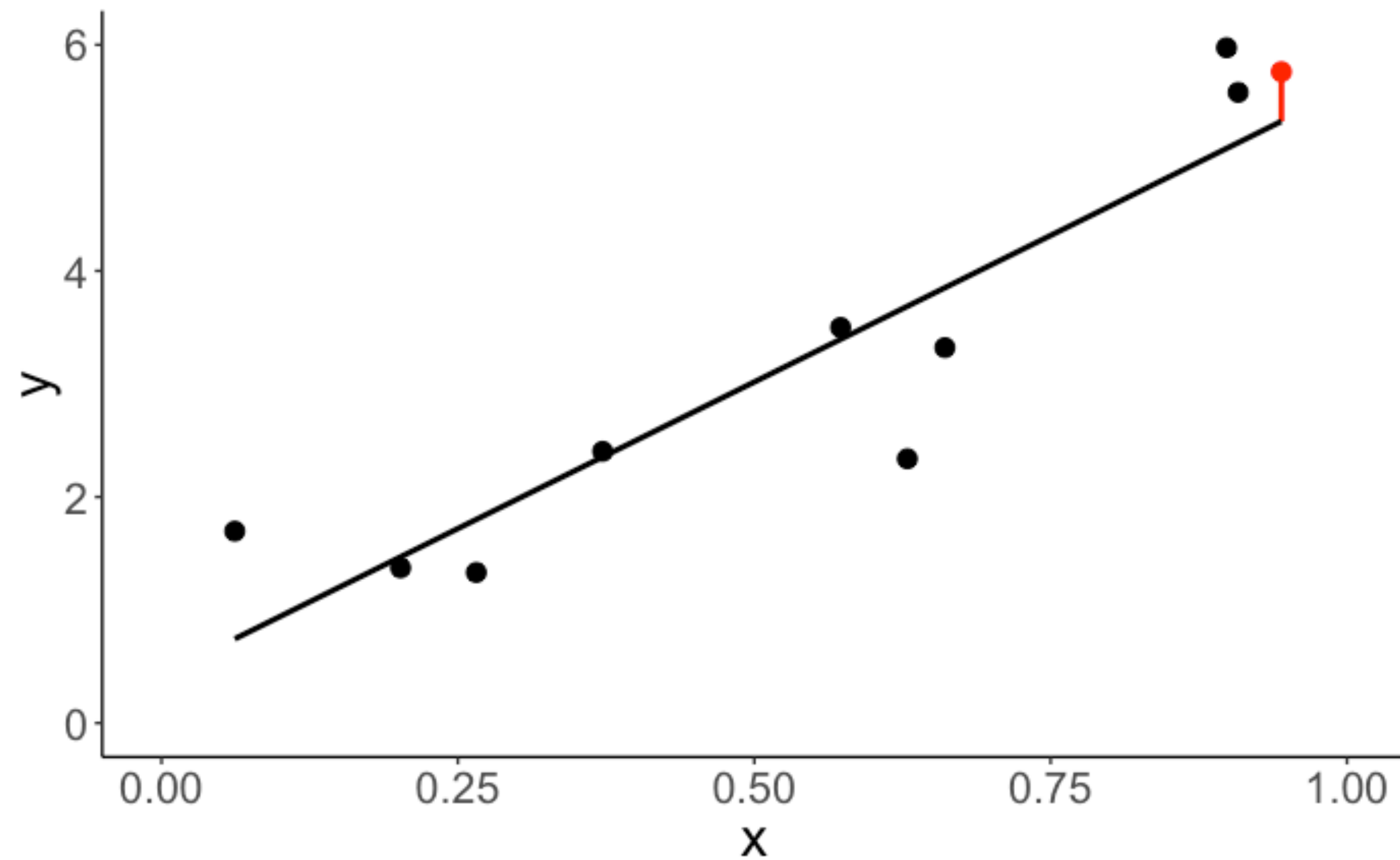
Leave one out cross-validation



Leave one out cross-validation

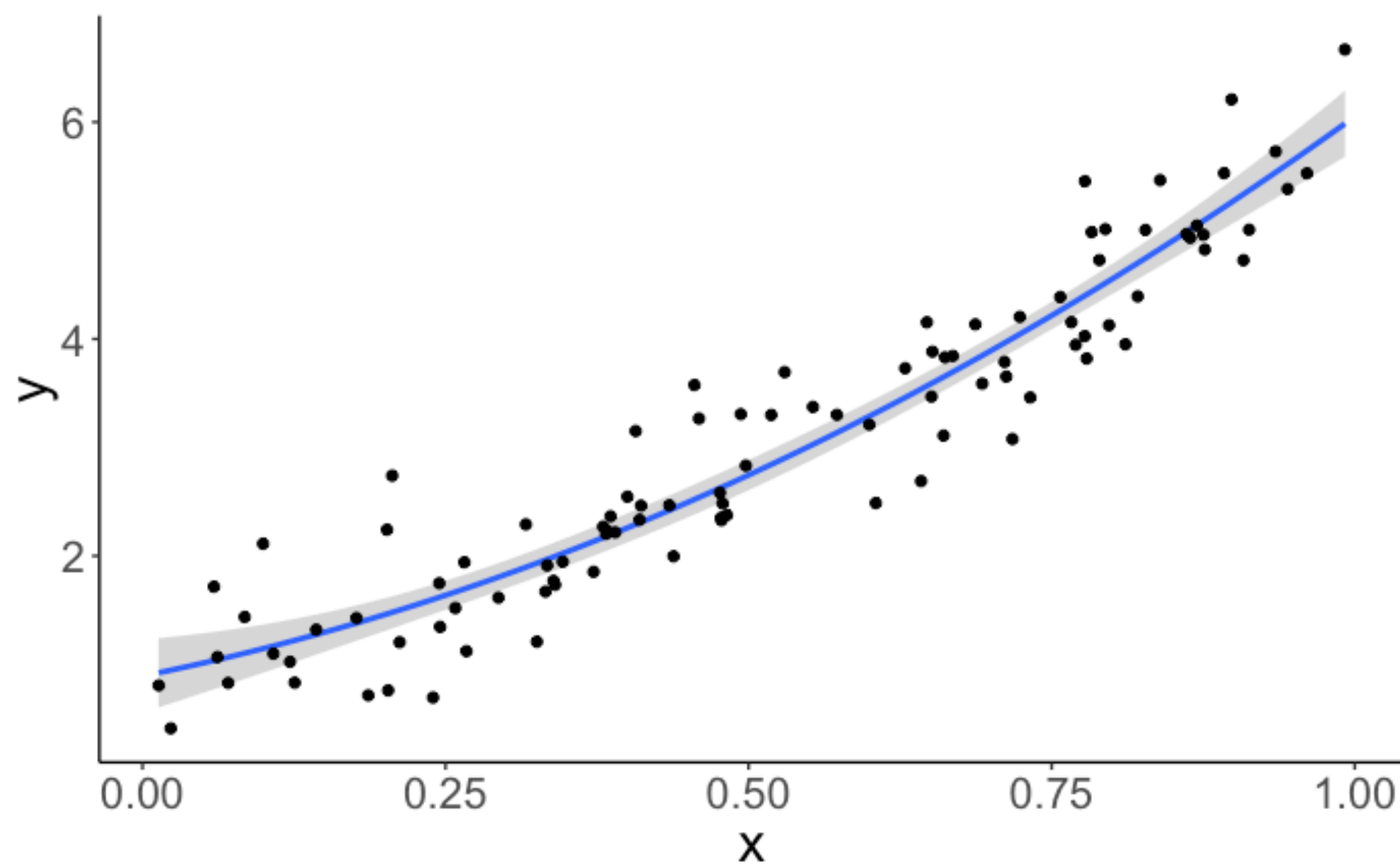


Leave one out cross-validation



Leave-one out crossvalidation

```
1 # make example reproducible
2 set.seed(1)
3
4 # parameters
5 sample_size = 100
6 b0 = 1
7 b1 = 2
8 b2 = 3
9 sd = 0.5
10
11 # sample
12 df.data = tibble(
13   participant = 1:sample_size,
14   x = runif(sample_size, min = 0, max = 1),
15   y = b0 + b1*x + b2*x^2 + rnorm(sample_size, sd = sd)
16 )
```



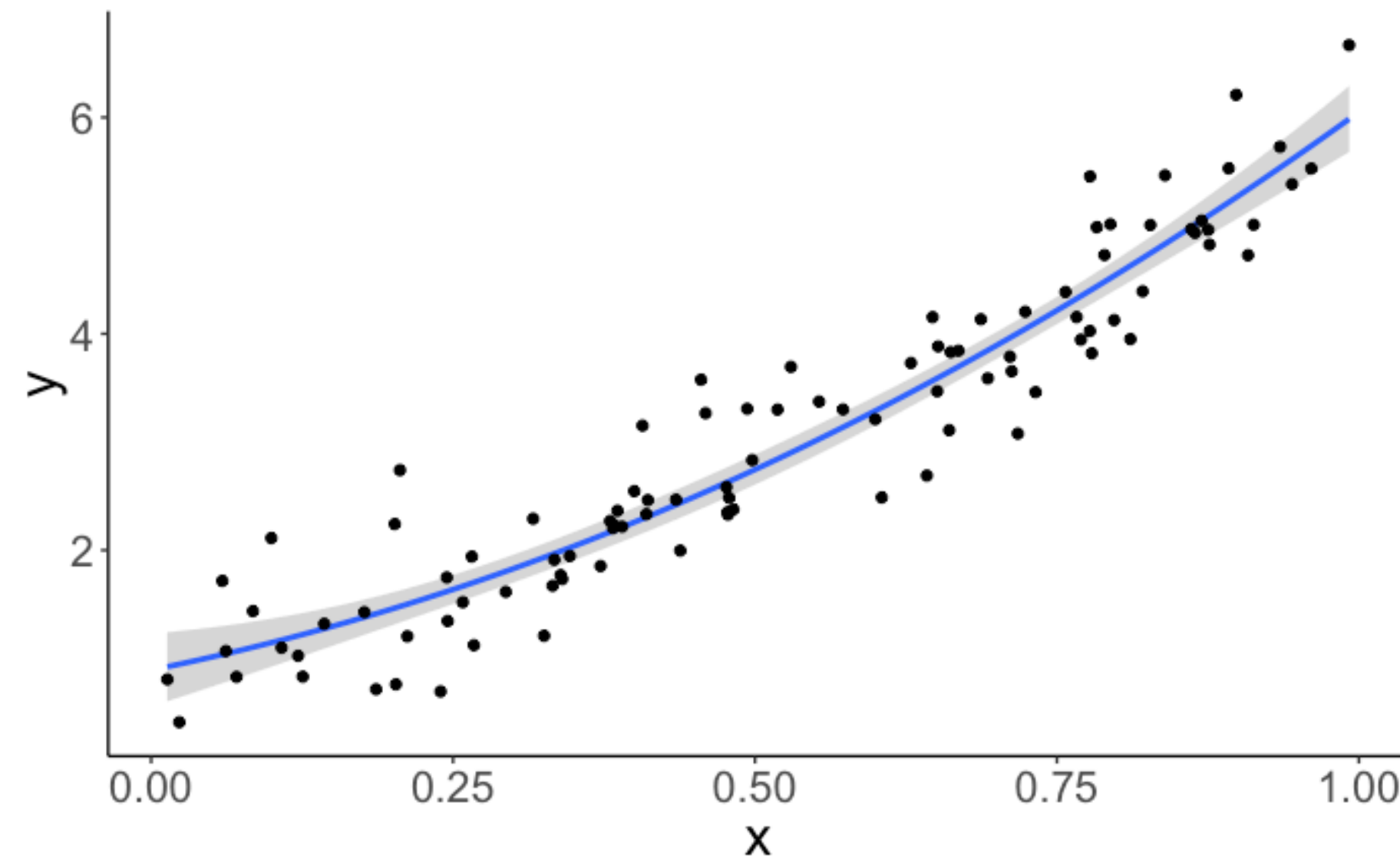
A scatter plot showing the relationship between x and y. The x-axis ranges from 0.00 to 1.00 with major ticks every 0.25. The y-axis ranges from 0 to 6 with major ticks every 2. The plot contains numerous black data points. A solid blue line represents the quadratic regression fit, starting at approximately (0, 1) and ending at (1, 6). A light gray shaded area around the blue line indicates the confidence interval, which widens as x increases.

Leave-one out crossvalidation

ground truth

$$y_i = 1 + 2 \cdot x_i + 3 \cdot x_i^2 + e$$
$$e \sim \mathcal{N}(\text{mean} = 0, \text{sd} = 0.5)$$

candidate models



simple $\hat{y}_i = b_0 + b_1 \cdot x_i$

correct $\hat{y}_i = b_0 + b_1 \cdot x_i + b_2 \cdot x_i^2$

complex $\hat{y}_i = b_0 + b_1 \cdot x_i + b_2 \cdot x_i^2 + b_3 \cdot x_i^3$

we could do an F-test
here since the models
are nested ...

Leave-one out crossvalidation

```
1 library("modelr") ← nice package for basic cross-validation
2
3 df.cross = df.data %>%
4   crossv_loo() %>% # function which generates training and test data sets
5   mutate(model_simple = map(train, ~ lm(y ~ 1 + x, data = .)),
6          model_correct = map(train, ~ lm(y ~ 1 + x + I(x^2), data = .)),
7          model_complex = map(train, ~ lm(y ~ 1 + x + I(x^2) + I(x^3), data = .))) %>%
8   pivot_longer(cols = contains("model"),
9               names_to = "index",
10              values_to = "model") %>%
11   mutate(rmse = map2_dbl(.x = model, .y = test, .f = ~ rmse(.x, .y)))
```

index	mean_rmse
simple	0.65
correct	0.42
complex	0.41

complex model has the lowest
error on the training data

Leave-one out crossvalidation

```
1 library("modelr")
2
3 df.cross = df.data %>%
4   crossv_loo() %>%
```

splits the data set into training and test

	train	test	.id
1	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	1
2	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	2
3	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	3
4	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	4
5	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	5
6	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	6
7	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	7
8	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	8
9	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	9
10	list(data = list(participant = 1:10, x = c(0.265508663...	list(data = list(participant = 1:10, x = c(0.265508663...	10

each entry is simply a pointer to the data set plus some indices `.idx` for the filtered data points

Leave-one out crossvalidation

```
1 library("modelr")
2
3 df.cross = df.data %>%
4   crossv_loo() %>%
5   mutate(model_simple = map(train, ~ lm(y ~ 1 + x, data = .)),
6          model_correct = map(train, ~ lm(y ~ 1 + x + I(x^2), data = .)),
7          model_complex = map(train, ~ lm(y ~ 1 + x + I(x^2) + I(x^3), data = .))) %>%
```

fit three different models to the training data set

Leave-one out crossvalidation

```
1 library("modelr")
2
3 df.cross = df.data %>%
4   crossv_loo() %>%
5   mutate(model_simple = map(train, ~ lm(y ~ 1 + x, data = .)),
6          model_correct = map(train, ~ lm(y ~ 1 + x + I(x^2), data = .)),
7          model_complex = map(train, ~ lm(y ~ 1 + x + I(x^2) + I(x^3), data = .))) %>%
8   pivot_longer(cols = contains("model"),
9               names_to = "index",
10              values_to = "model") %>%
11   mutate(rmse = map2_dbl(.x = model, .y = test, .f = ~ rmse(.x, .y)))
```

**calculate the root mean squared error for
each model on the test data set**

```
1 df.cross %>%
2   group_by(index) %>%
3   summarize(mean_rmse = mean(rmse))
```

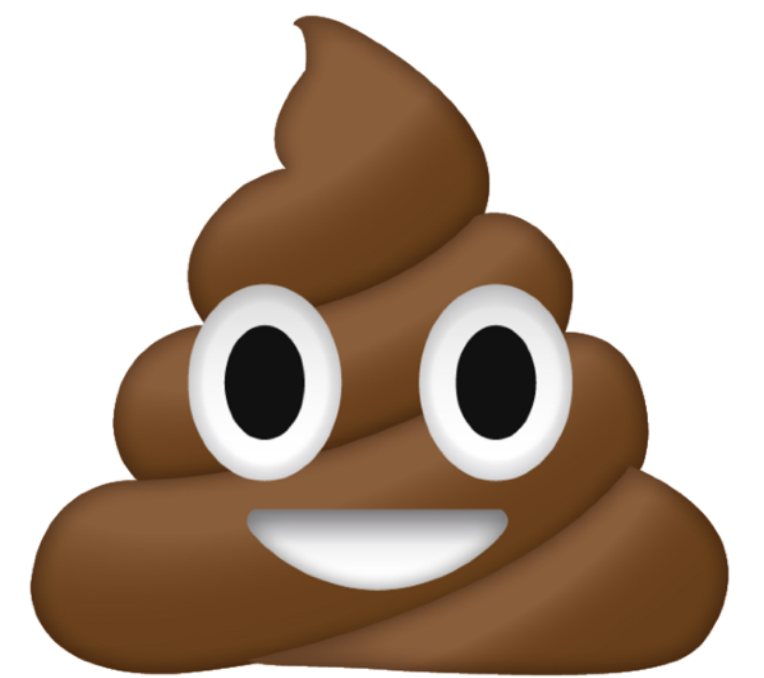
index	mean_rmse
simple	0.65
correct	0.48
complex	0.70

**the correct model has the
lowest prediction error**

Leave-one out crossvalidation

Any potential problems with the LOO?

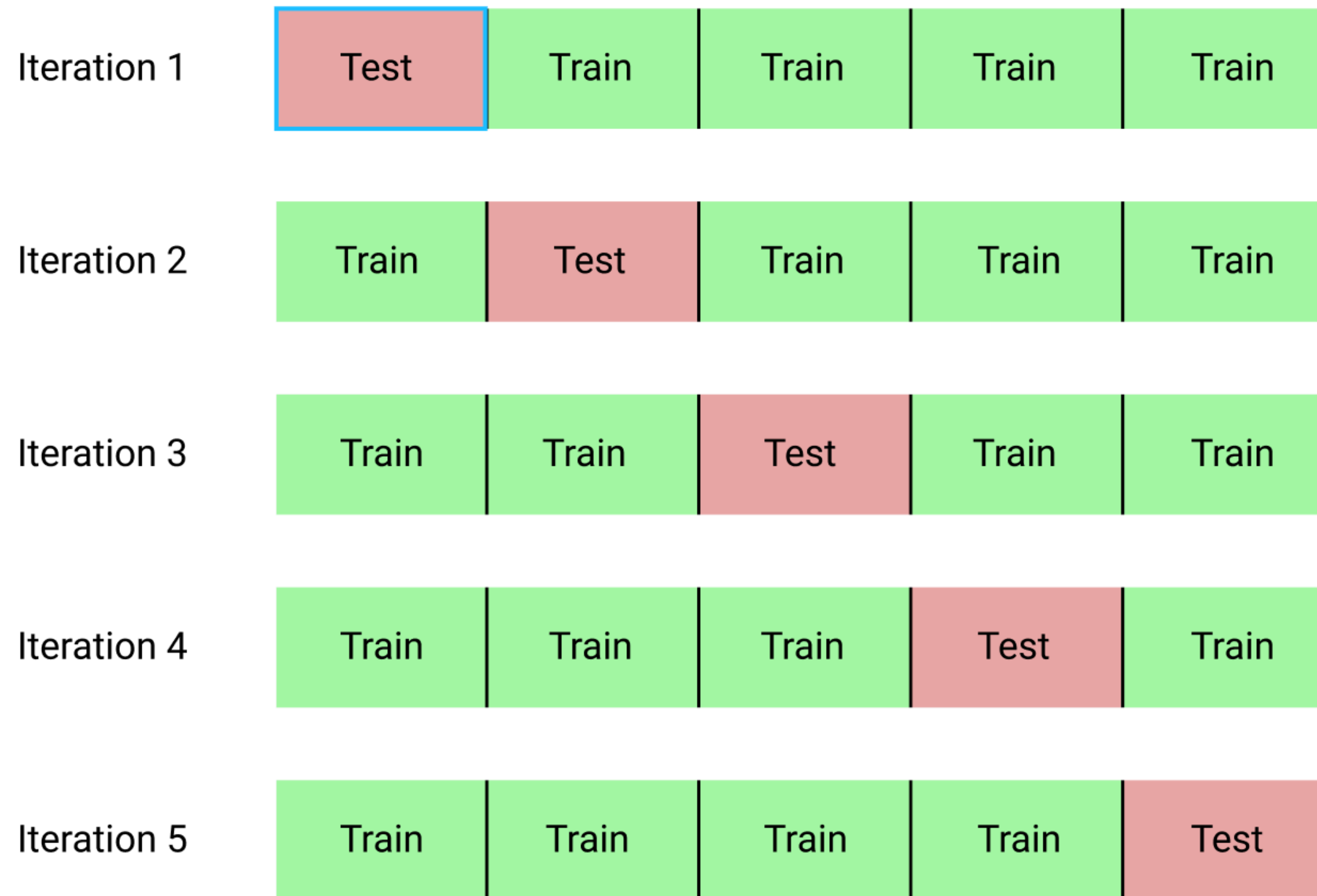
can be computationally expensive since it requires fitting the model n times (once for each data point) ...



k-fold cross validation

k-fold crossvalidation

Full data set



k-fold crossvalidation

k-fold crossvalidation

```
1 df.cross = df.data %>%  
2   crossv_kfold(k = 10) %>%  
3   mutate(model_simple = map(.x = train,  
4     .f = ~ lm(y ~ 1 + x, data = .)),  
5     model_correct = map(.x = train,  
6       .f = ~ lm(y ~ 1 + x + I(x^2), data = .)),  
7     model_complex = map(.x = train,  
8       .f = ~ lm(y ~ 1 + x + I(x^2) + I(x^3), data = .))) %>%  
9   pivot_longer(cols = contains("model"),  
10     names_to = "model",  
11     values_to = "fit") %>%  
12   mutate(rsquare = map2_dbl(.x = fit,  
13     .y = test,  
14     .f = ~ rsquare(.x, .y)))
```

using R^2 as a measure

why didn't we use R^2 for LOO?

this wouldn't work for LOO
since we only have one data
point in the test data...

index	median_rsquare
simple	0.839
correct	0.865
complex	0.860

**the correct model accounts for
the most variance in the test data**

k-fold vs. leave-one-out crossvalidation

- **LOO:**

- trained on **more** data
- more variance
- less bias

more overfitting

less underfitting

- **k-fold:**

- trained on **less** data
- less variance
- more bias

less overfitting

more underfitting

- **bias** = error from erroneous model assumptions, miss the relevant relations
- **variance** = error from small fluctuations in the data, fitting the random noise

Monte Carlo crossvalidation

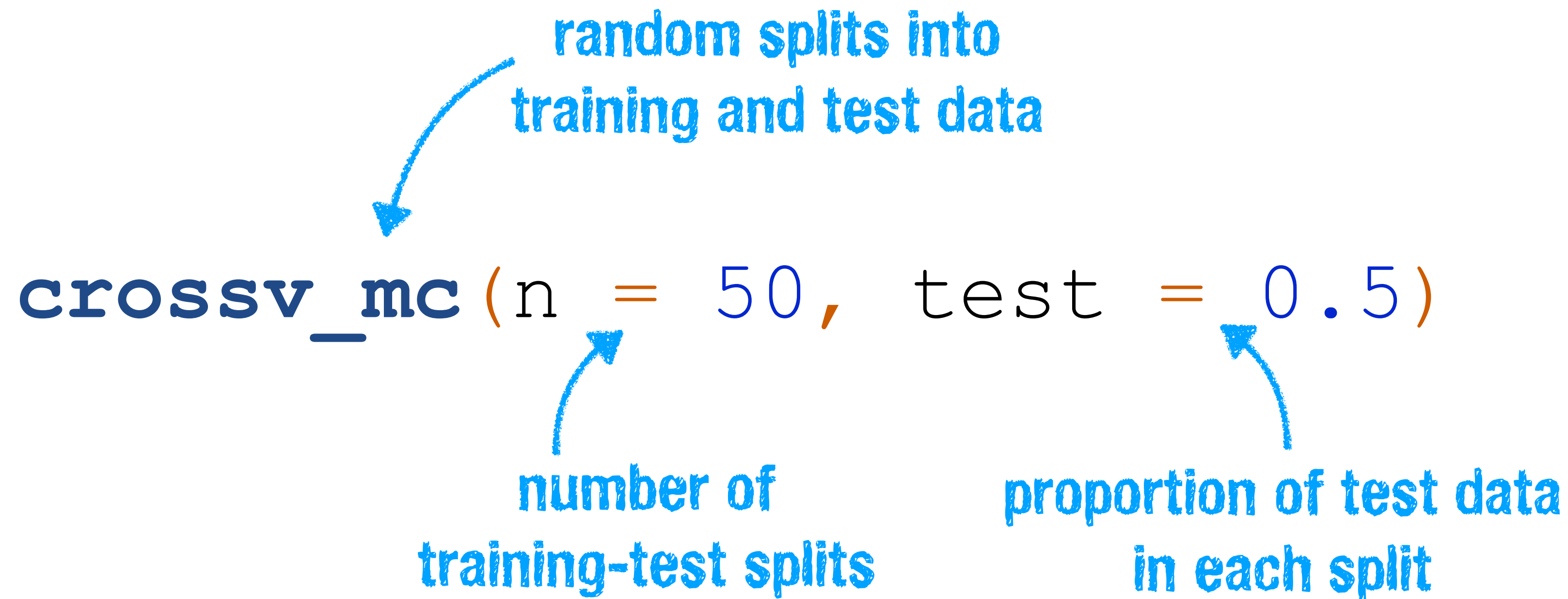
Monte Carlo crossvalidation

`crossv_mc` (n = 50, test = 0.5)

random splits into training and test data

number of training-test splits

proportion of test data in each split

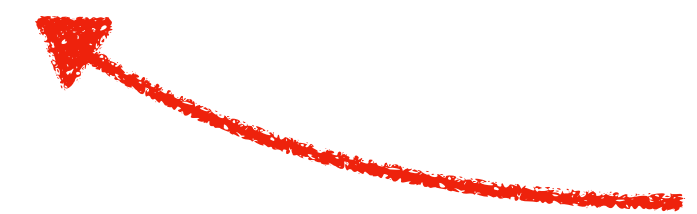


- splits can also be done in a **stratified** way (check out the `tidymodels` package)
- for example, generate training data that has the same percentage of cases from each group
- fit data from some participants, test on data from other participants, ...

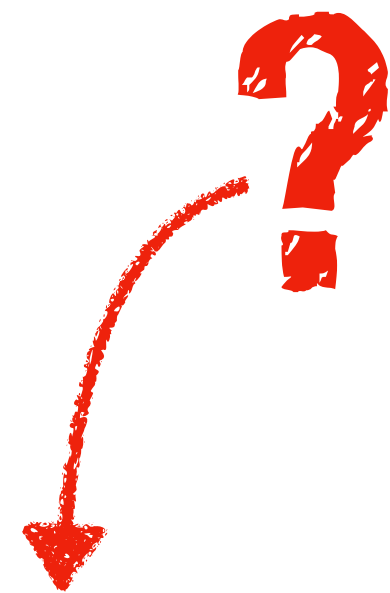
AIC and BIC

AIC and BIC

- AIC = Akaike Information Criterion
- BIC = Bayesian Information Criterion

 **not that much Bayesian about it ...**

$$\text{AIC} = 2k - 2 \ln(\hat{L})$$



$$\text{BIC} = \ln(n)k - 2 \ln(\hat{L})$$

$\hat{L}$ = maximized value of the likelihood function of the model

k = number of parameters in the model

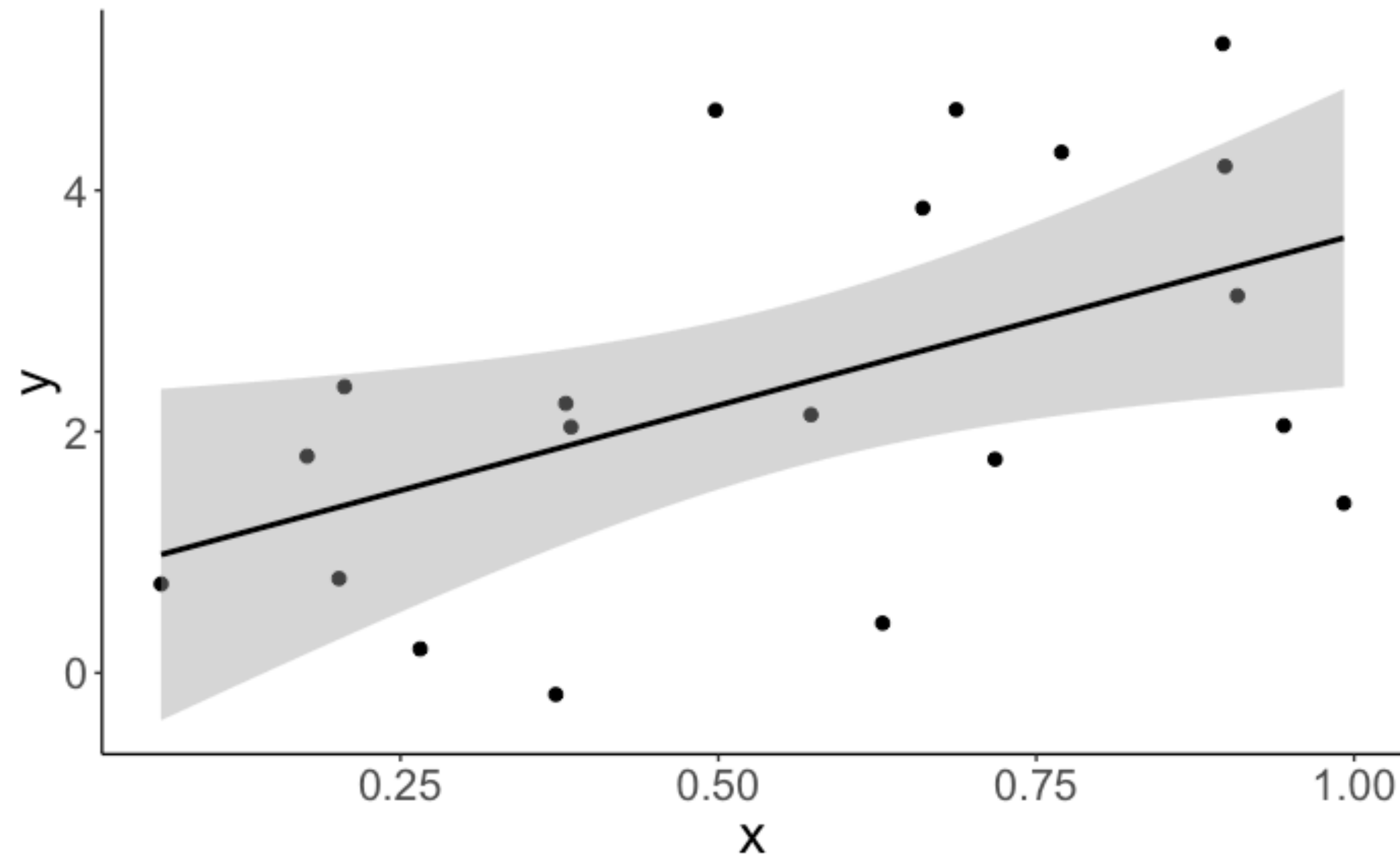
n = number of observations

AIC and BIC

- How do we get the likelihood of our model?
 - in a linear regression, minimizing least squares is equivalent to maximizing the likelihood of the data given the model
- Assumptions of the linear model:
 - residuals are normally distributed with:
 - $\text{mean} = 0$ and $\text{sd} = \text{sigma}$
 - calculate overall likelihood by computing the likelihood of each residual, and then multiplying

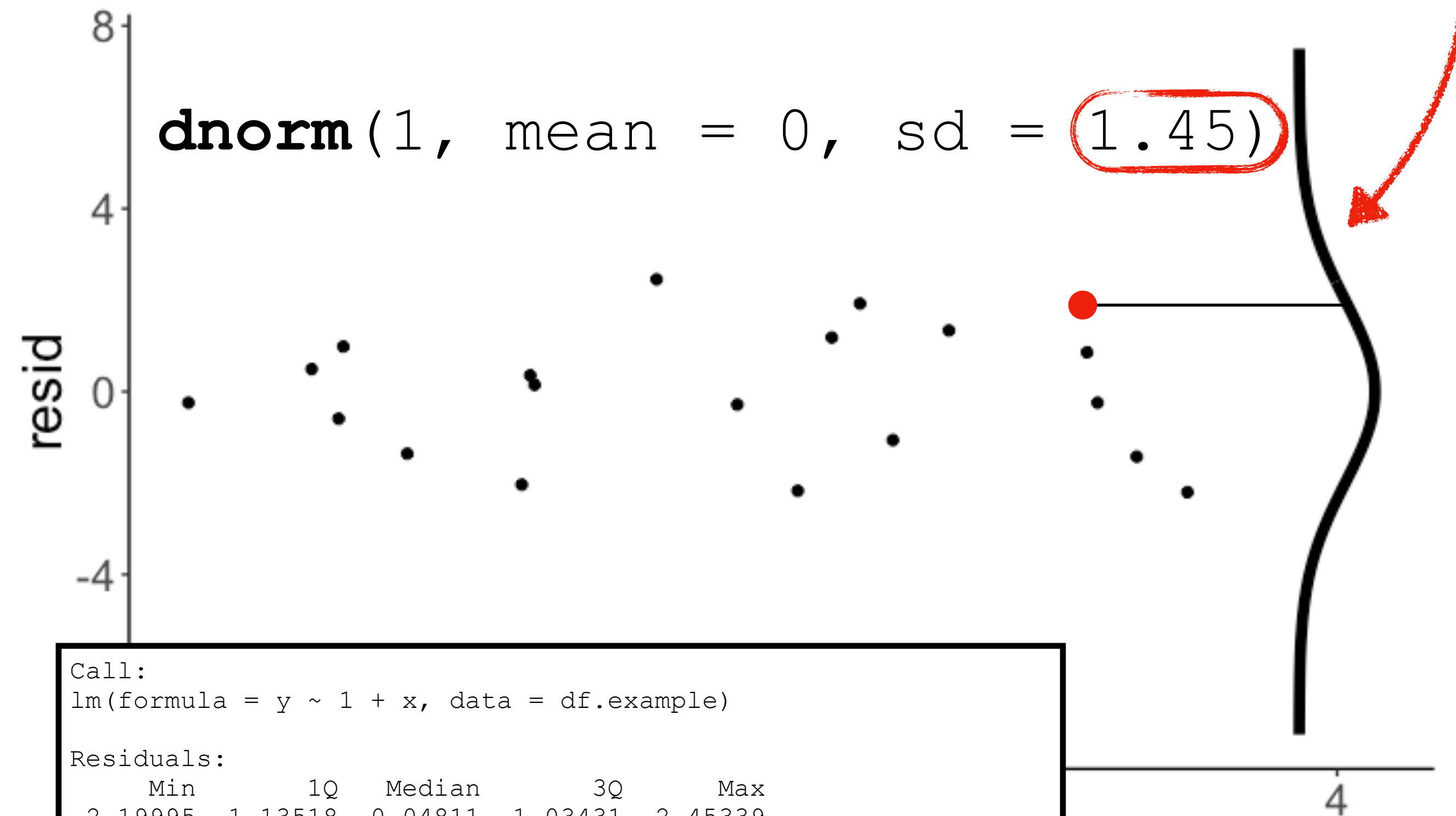
AIC and BIC

data with linear model fit



AIC and BIC

Residual plot



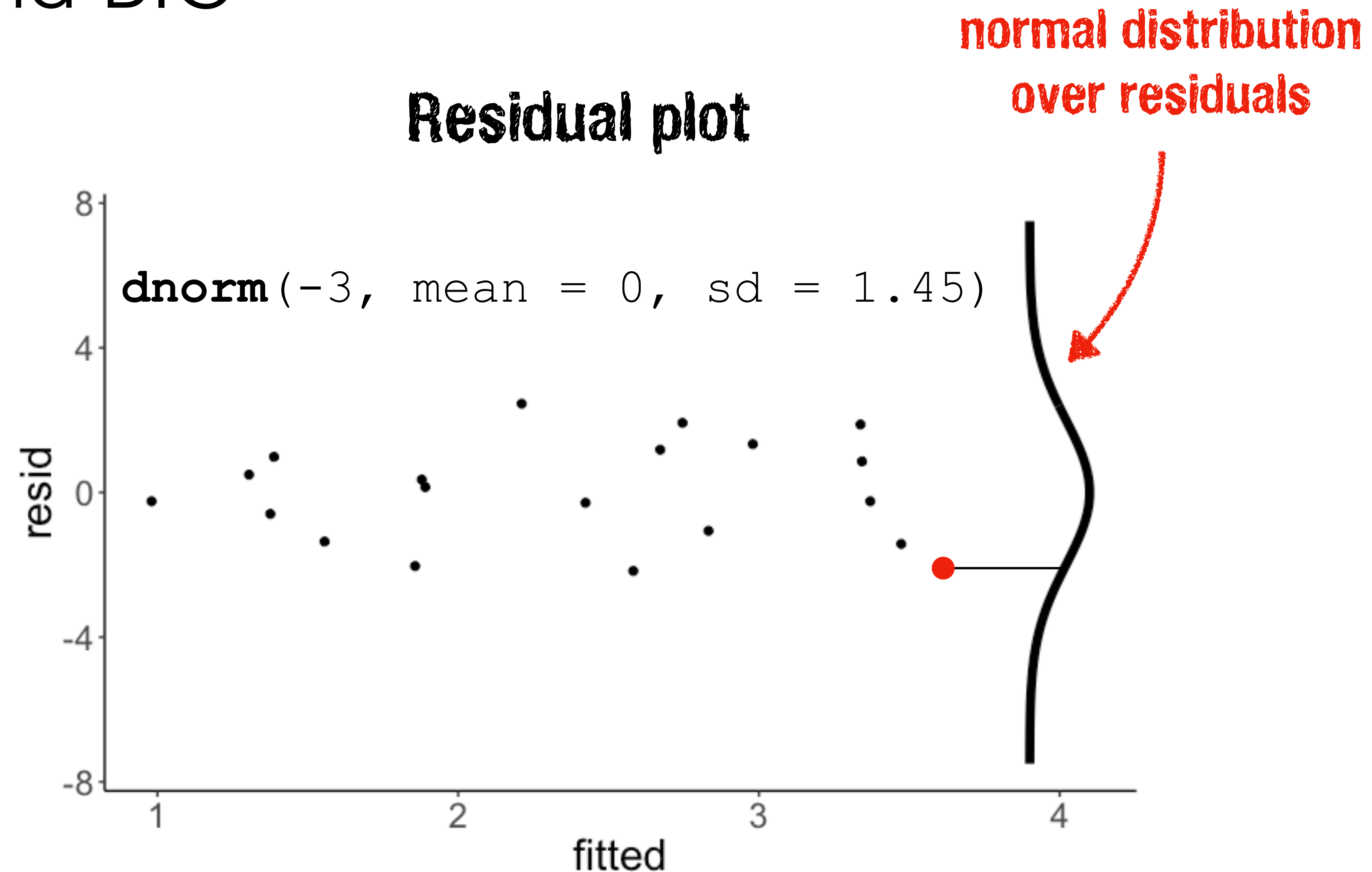
```
Call:
lm(formula = y ~ 1 + x, data = df.example)

Residuals:
    Min       1Q   Median       3Q      Max
-2.19995 -1.13518 -0.04811  1.03431  2.45339

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.8062     0.7157   1.126   0.2748
x            2.8232     1.1374   2.482   0.0232 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.449 on 18 degrees of freedom
Multiple R-squared:  0.255,    Adjusted R-squared:  0.2136
F-statistic: 6.161 on 1 and 18 DF,  p-value: 0.02315
```

AIC and BIC



since the data points are independent, we can calculate the overall likelihood by multiplying the likelihood of each observation

AIC and BIC

```
dnorm(1.88, mean = 0, sd = 1.45) = 0.12
```

```
1 # generate some data
2 df.like = tibble(
3   x = runif(20, min = 0, max = 1),
4   y = 1 + 3 * x + rnorm(20, sd = 2)
5 )
6
7 # fit the model
8 fit = lm(formula = y ~ x,
9          data = df.like)
10
11 # model summary
12 fit %>%
13   glance()
```

x	y	fitted	resid	likelihood
0.90	5.22	3.34	1.88	0.12
0.27	0.20	1.56	-1.36	0.18
0.37	-0.18	1.86	-2.04	0.10
0.57	2.14	2.42	-0.28	0.27
0.91	3.13	3.37	-0.24	0.27
0.20	0.78	1.38	-0.59	0.25
0.90	4.20	3.34	0.86	0.23
0.94	2.05	3.47	-1.42	0.17
0.66	3.85	2.67	1.18	0.20
0.63	0.41	2.58	-2.17	0.09

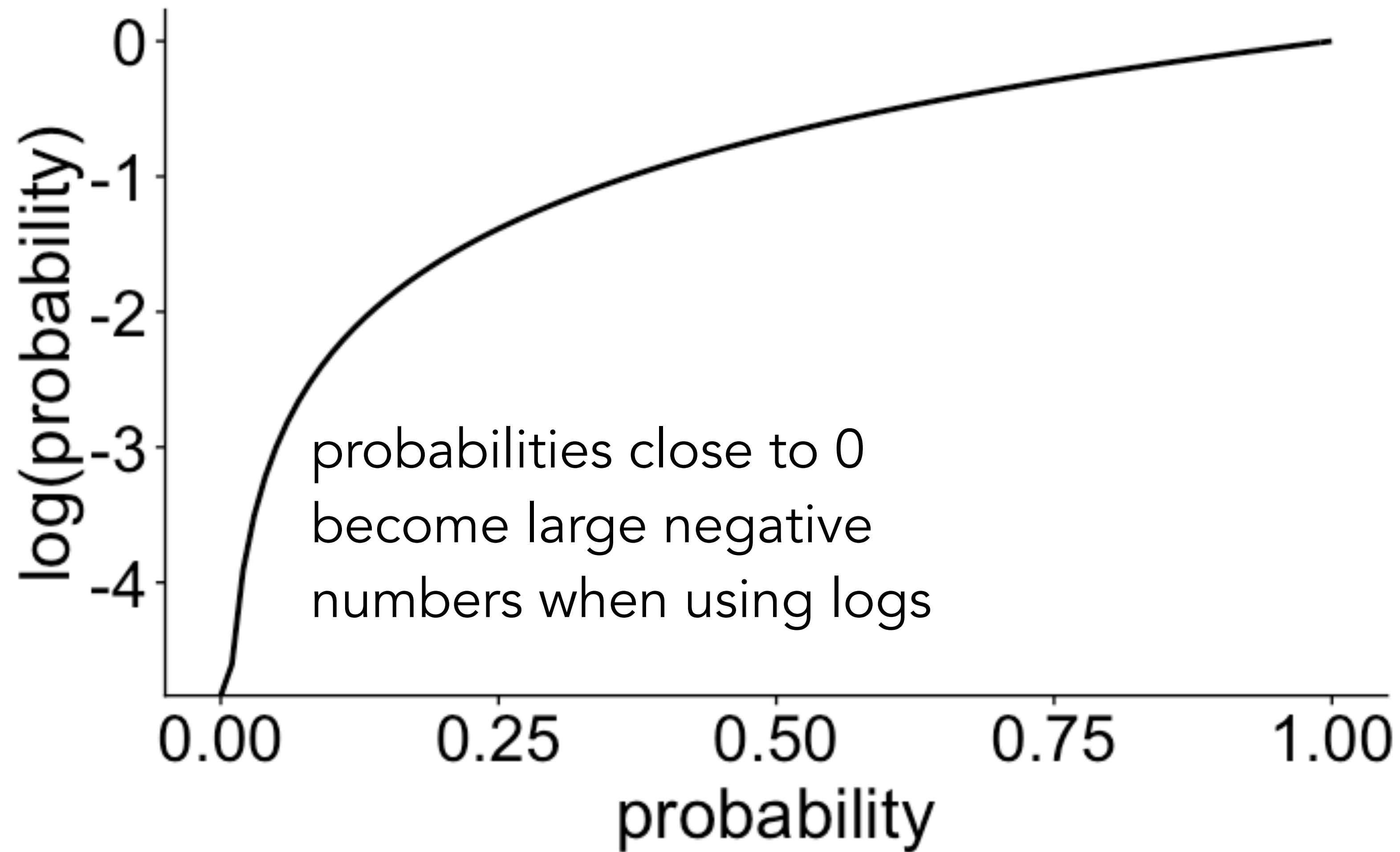
inferred standard
deviation of the error

r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual
0.25	0.21	1.45	6.16	0.02	2	-34.74	75.47	78.46	37.77	18

$$\sum_{i=1}^n \ln(\text{likelihood})$$

$e \sim \mathcal{N}(\text{mean} = 0, \text{sd} = 1.45)$

$\log()$ is your friend!



Clue guide to probability

- *joint probability:*
- if A and B are independent then
- **Definition:** $p(A, B) = p(A) \cdot p(B)$
- $p(\text{Prof Plum, candle stick}) =$
 $p(\text{Prof Plum}) \cdot p(\text{candle stick}) =$
 $\frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$

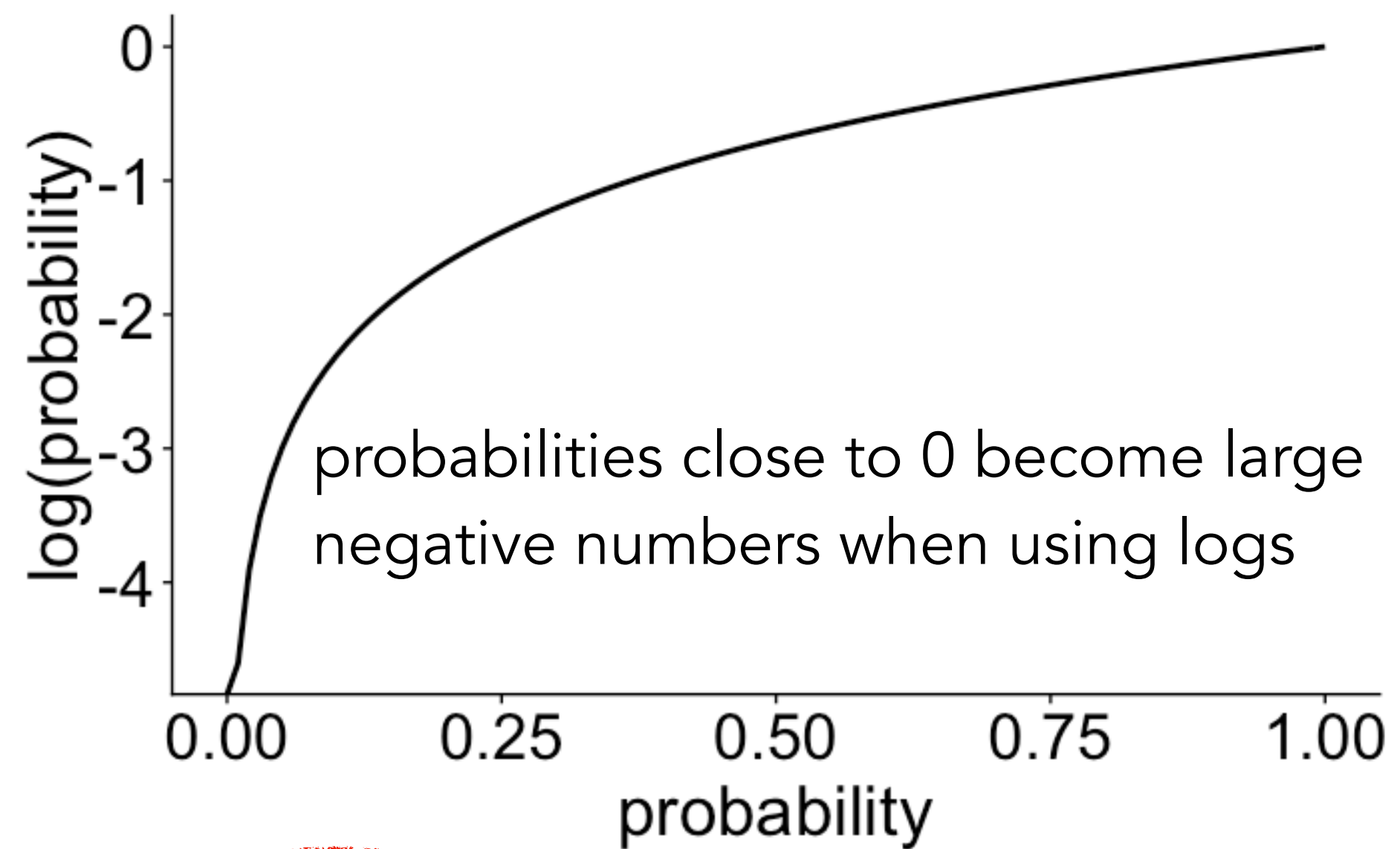
Who?



What?



`log ( )` is your friend!



multiplying probabilities

$$0.01 \cdot 0.01 \cdot 0.01 \cdot 0.01 = 0.00000001$$

number becomes
extremely small quickly

take `log()`

$$\log(0.01) = -4.60517$$

number becomes large
but that's ok

summing logs

$$(-4.60517) + (-4.60517) + (-4.60517) + (-4.60517) = -18.42068$$

transform back into probability

$$\exp(-18.42068) = 0.00000001$$

often not necessary since
we just use logLikelihood

AIC and BIC

```
dnorm(1.88, mean = 0, sd = 1.45) = 0.12
```

```
1 # generate some data
2 df.like = tibble(
3   x = runif(20, min = 0, max = 1),
4   y = 1 + 3 * x + rnorm(20, sd = 2)
5 )
6
7 # fit the model
8 fit = lm(formula = y ~ x,
9          data = df.like)
10
11 # model summary
12 fit %>%
13   glance()
```

x	y	fitted	resid	likelihood
0.90	5.22	3.34	1.88	0.12
0.27	0.20	1.56	-1.36	0.18
0.37	-0.18	1.86	-2.04	0.10
0.57	2.14	2.42	-0.28	0.27
0.91	3.13	3.37	-0.24	0.27
0.20	0.78	1.38	-0.59	0.25
0.90	4.20	3.34	0.86	0.23
0.94	2.05	3.47	-1.42	0.17
0.66	3.85	2.67	1.18	0.20
0.63	0.41	2.58	-2.17	0.09

inferred standard
deviation of the error

$$\sum_{i=1}^n \ln(\text{likelihood})$$

r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual
0.25	0.21	1.45	6.16	0.02	2	-34.74	75.47	78.46	37.77	18

$e \sim \mathcal{N}(\text{mean} = 0, \text{sd} = 1.45)$

AIC and BIC

$$\text{AIC} = 2k - 2 \ln(\hat{L})$$

$$\text{BIC} = \ln(n)k - 2 \ln(\hat{L})$$

$\ln(\hat{L})$ = maximized value of the likelihood function of the model **-34.74**

k = number of parameters in the model **3**

n = number of observations **20**

the sd of the normal
distribution modeling the
residuals counts as a parameter

```
lm(formula = y ~ 1 + x, data = df.example)
```

r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual
0.25	0.21	1.45	6.16	0.02	2	-34.74	75.47	78.46	37.77	18

AIC and BIC

$$\text{AIC} = 2k - 2 \ln(\hat{L}) = 2 \cdot 3 - 2 \cdot (-34.74) = 75.47$$

$$\text{BIC} = \ln(n)k - 2 \ln(\hat{L}) = \ln(20) \cdot 3 - 2 \cdot (-34.74) = 78.46$$

$\ln(\hat{L})$ = maximized value of the likelihood function of the model **-34.74**

k = number of parameters in the model **3**

n = number of observations **20**

the sd of the normal
distribution modeling the
residuals counts as a parameter

```
lm(formula = y ~ 1 + x, data = df.example)
```

r.squared	adj.r.squared	sigma	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual
0.25	0.21	1.45	6.16	0.02	2	-34.74	75.47	78.46	37.77	18

AIC and BIC

$$AIC = 2k - 2 \ln(\hat{L})$$

$$BIC = \ln(n)k - 2 \ln(\hat{L})$$

- for both AIC and BIC, *lower* is better!
- neither provide a test of a model in the sense of testing a null hypothesis
 - AIC or BIC tell us nothing about the absolute quality of a model, only the quality relative to other models
- BIC generally penalizes free parameters more strongly than AIC (though it depends on the size of n)

ΔBIC	Evidence against higher BIC
0 to 2	Not worth more than a bare mention
2 to 6	Positive
6 to 10	Strong
>10	Very Strong

What shall I use when?

- Use it all!
- ideally, the different measures provide converging evidence

Table 2

Summary of the model results. Values for r and RMSE indicate means (with 5% and 95% quantiles in parentheses) based on 100 split-half cross-validation runs. BIC scores are based on running the models on the full data set.

Model	r	RMSE	BIC
Difference & pivotality	.86 (.66, .95)	10.56 (6.17, 17.21)	158.59
Difference	.70 (.30, .90)	26.92 (16.4, 40.6)	209.74
Pivotality	.63 (.41, .77)	14.23 (11.39, 17.54)	199.53
Optimality	.66 (.42, .84)	14.55 (10.54, 17.91)	199.47

Note: BIC = Bayesian Information Criterion (lower values indicate better model performance).

Summary

- Quick recap
- Power analysis
 - Making decisions
 - Calculating power
 - Effect sizes
 - Determining sample size
- Model comparison
 - Cross-validation
 - AIC and BIC

How was the pace of today's class?

much too slow a little too slow just right a little too fast much too fast

How happy were you with today's class overall?



What did you like about today's class? What could be improved next time?

Thank you!